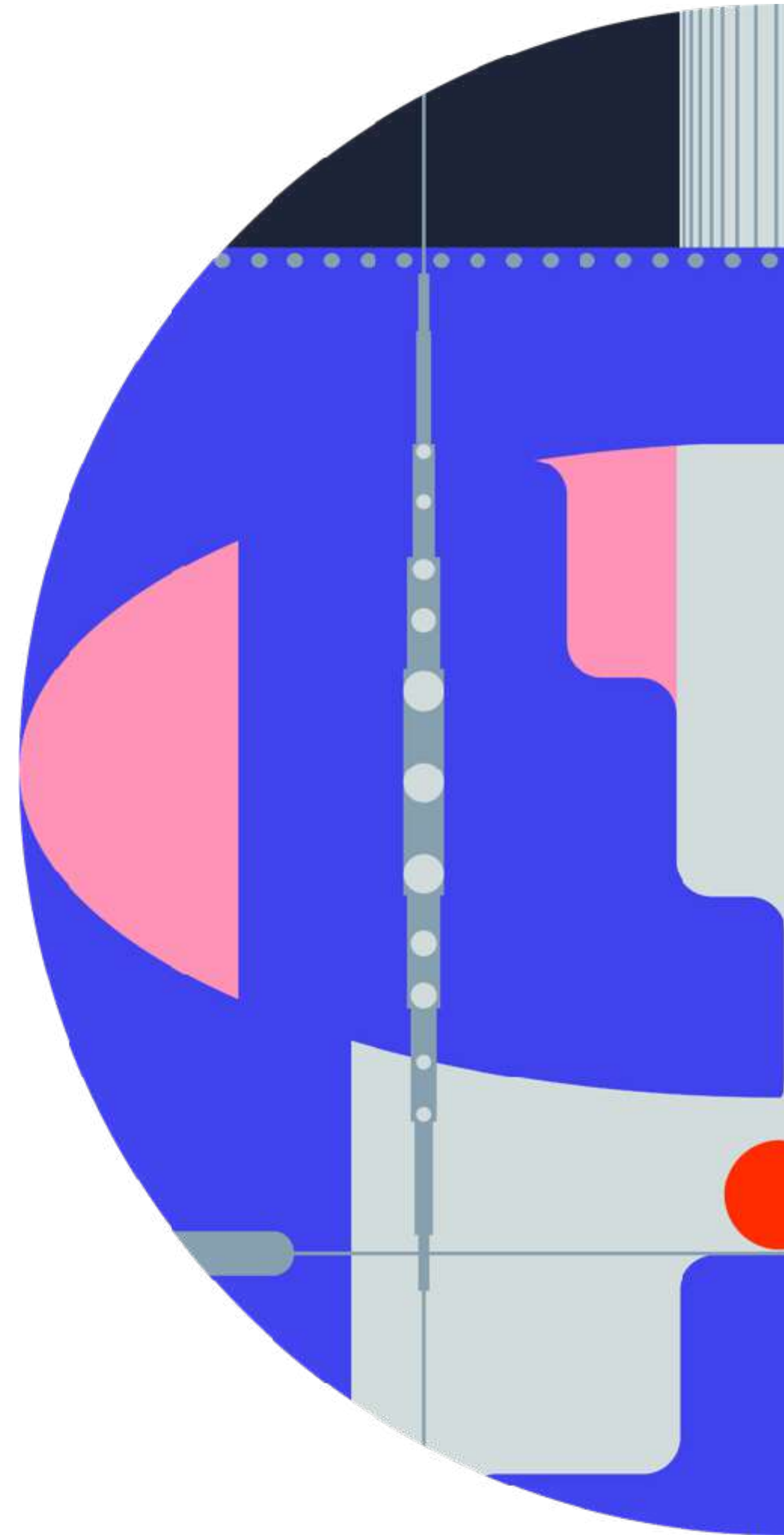


Diffusion models in practice



Outline

01

“Quick” recap

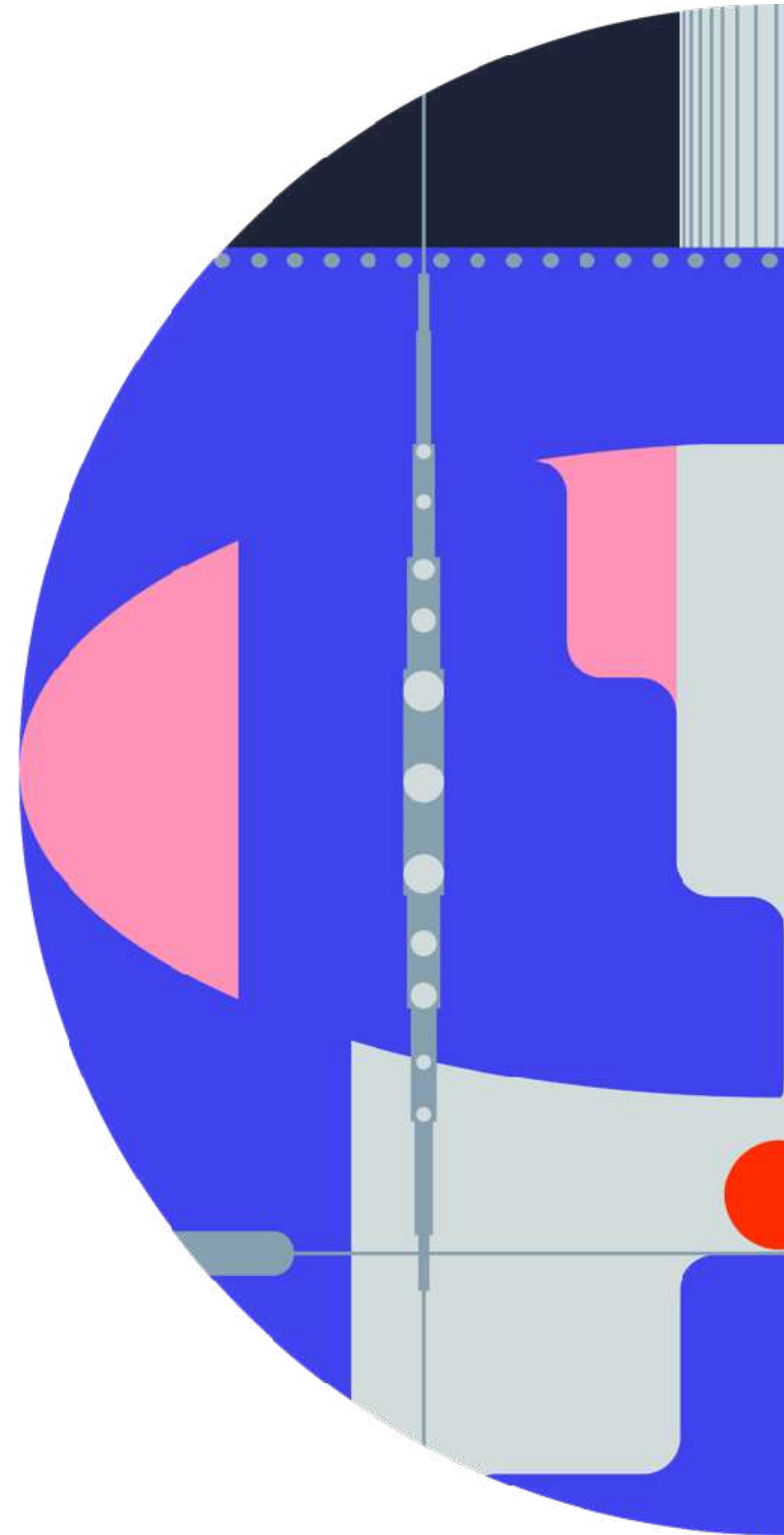
02

Designing and training diffusion models

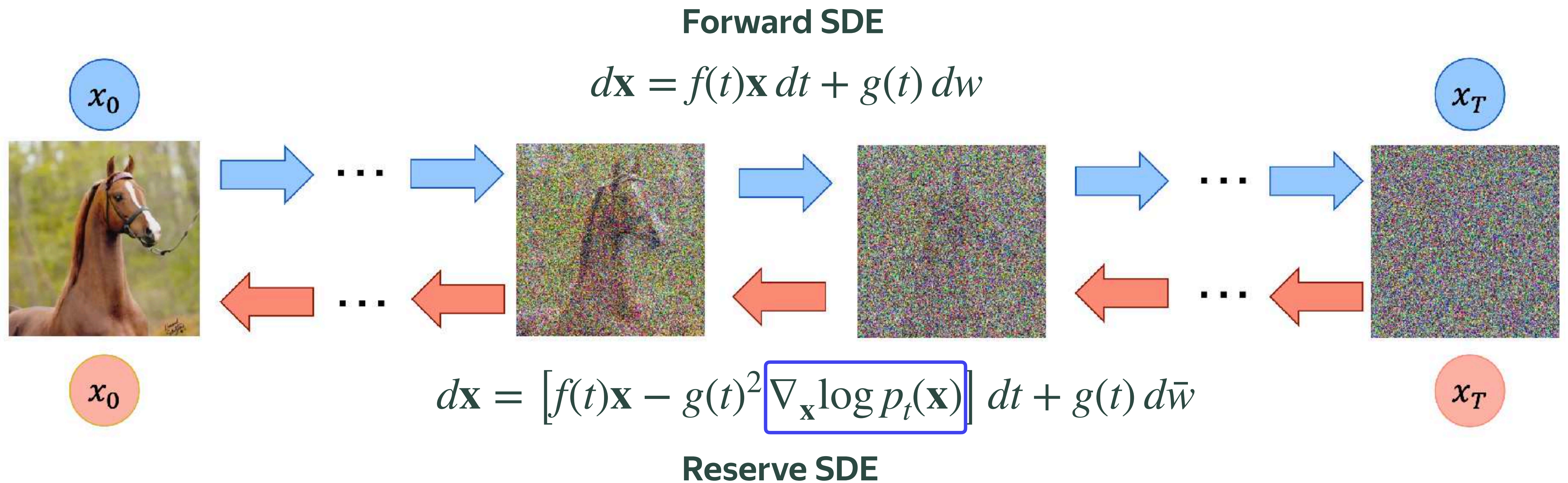
03

Advanced sampling techniques in diffusion models

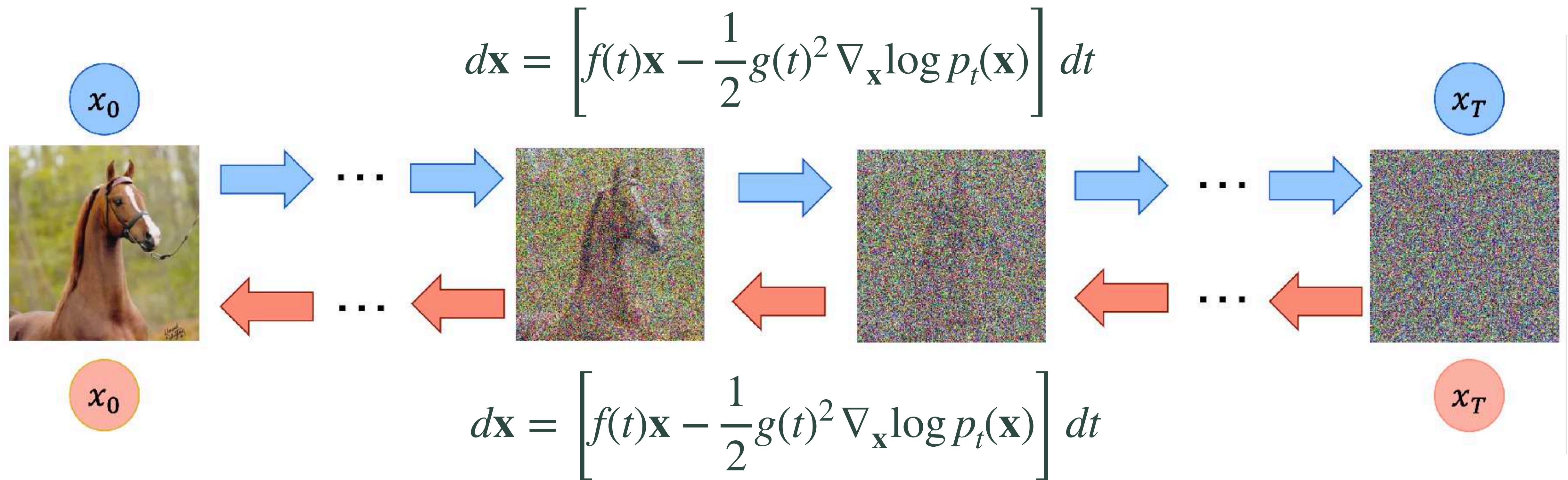
“Quick” recap on diffusion models



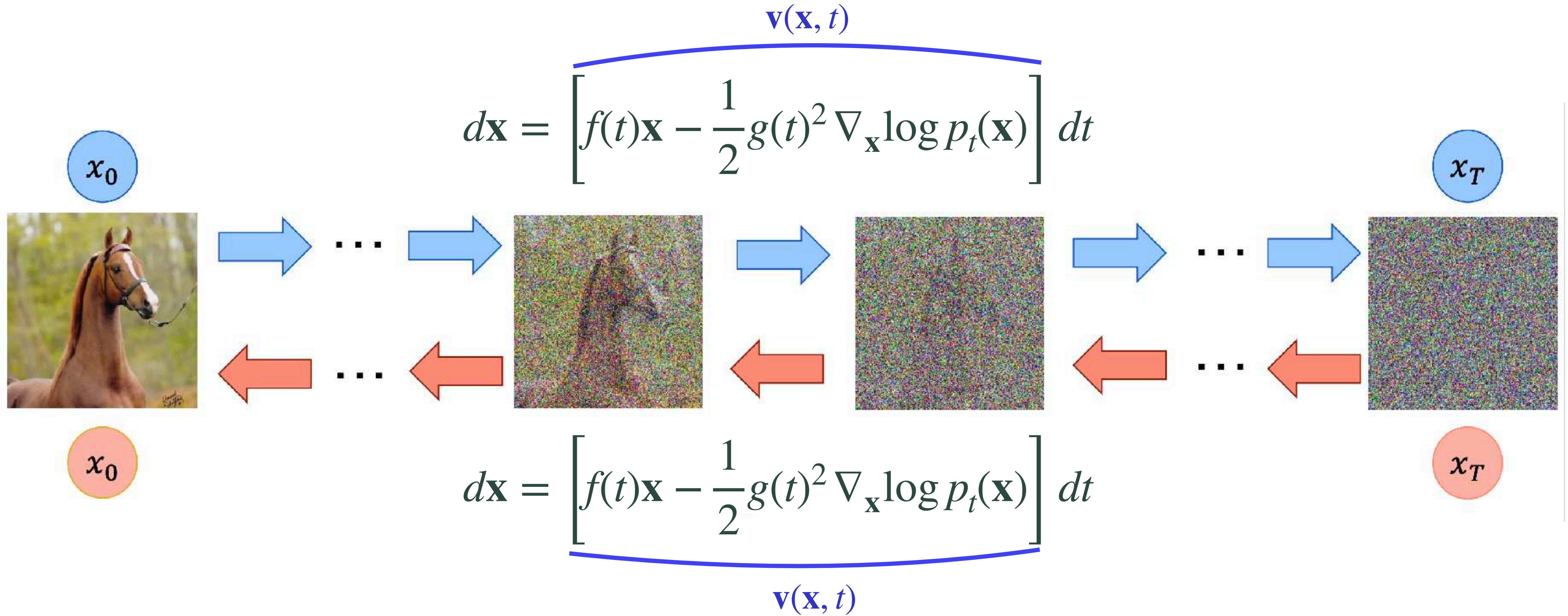
Stochastic differential equations



Probability Flow ODE



Flow matching



Forward process

General Forward Process: $p(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\alpha_t \mathbf{x}_0, \sigma_t^2 I)$

$$\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Variance Preserving (VP): $\alpha_t^2 + \sigma_t^2 = 1$

DDPM linear- β : $\alpha_t = e^{-\frac{1}{2} \int_0^t \beta(s) ds}$ **Cosine:** $\alpha_t = \cos\left(\frac{\pi t}{2}\right), \quad \sigma_t = \sin\left(\frac{\pi t}{2}\right)$

Variance Exploding (VE): $\alpha_t = 1, \quad \sigma_t = t$

Linear Interpolation (Flow Matching): $\alpha_t = 1 - t, \quad \sigma_t = t$

Training

Score Matching

Tweedie's formula

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log \int p_{data}(\mathbf{x}) p(\mathbf{x}_t | \mathbf{x}) d\mathbf{x} = \mathbb{E} \left[\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{x}_0) \mid \mathbf{x}_t \right] \quad \text{Intractable target}$$

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{x}_0) = -\frac{1}{\sigma_t^2} (\mathbf{x}_t - \alpha_t \mathbf{x}_0) = -\frac{\epsilon}{\sigma_t}$$

Conditional trick

$$\mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[w(t) \cdot \|\mathbf{s}_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)\|_2^2 \right] \overset{\text{Conditional trick}}{=} \mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[w(t) \cdot \|\mathbf{s}_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{x}_0)\|_2^2 \right] + C$$

Training

Flow Matching

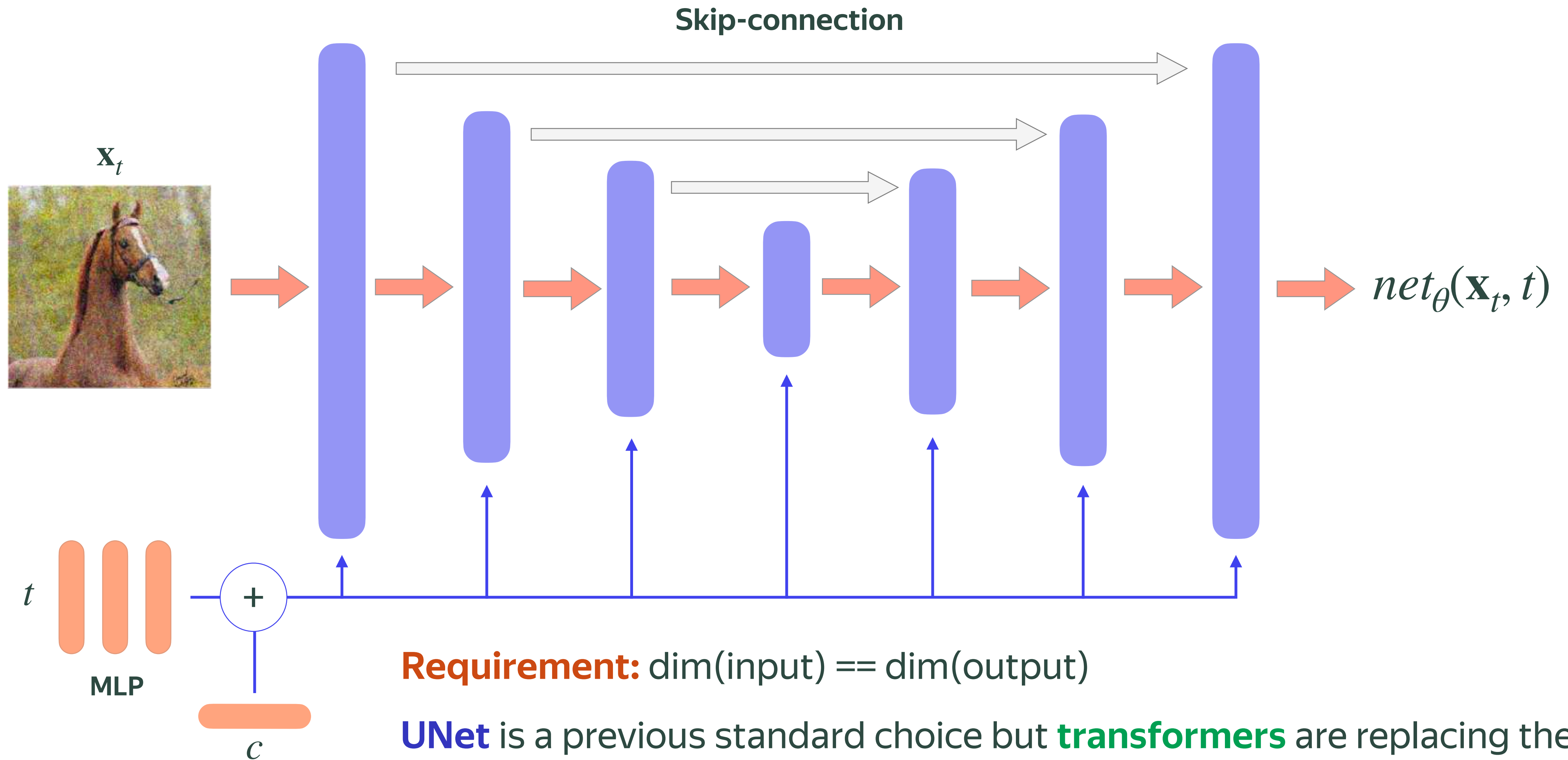
$$\mathbf{v}(\mathbf{x}_t) := \mathbb{E} \left[\mathbf{v}^{\text{cond}}(\mathbf{x}_t) \mid \mathbf{x}_t \right] \quad \text{Intractable target}$$

$$\mathbf{v}^{\text{cond}}(\mathbf{x}_t) := \dot{\mathbf{x}}_t = \dot{\alpha}_t \mathbf{x}_0 + \dot{\sigma}_t \boldsymbol{\epsilon} = \{ \alpha_t = (1 - t), \sigma_t = t \} = \boldsymbol{\epsilon} - \mathbf{x}_0$$

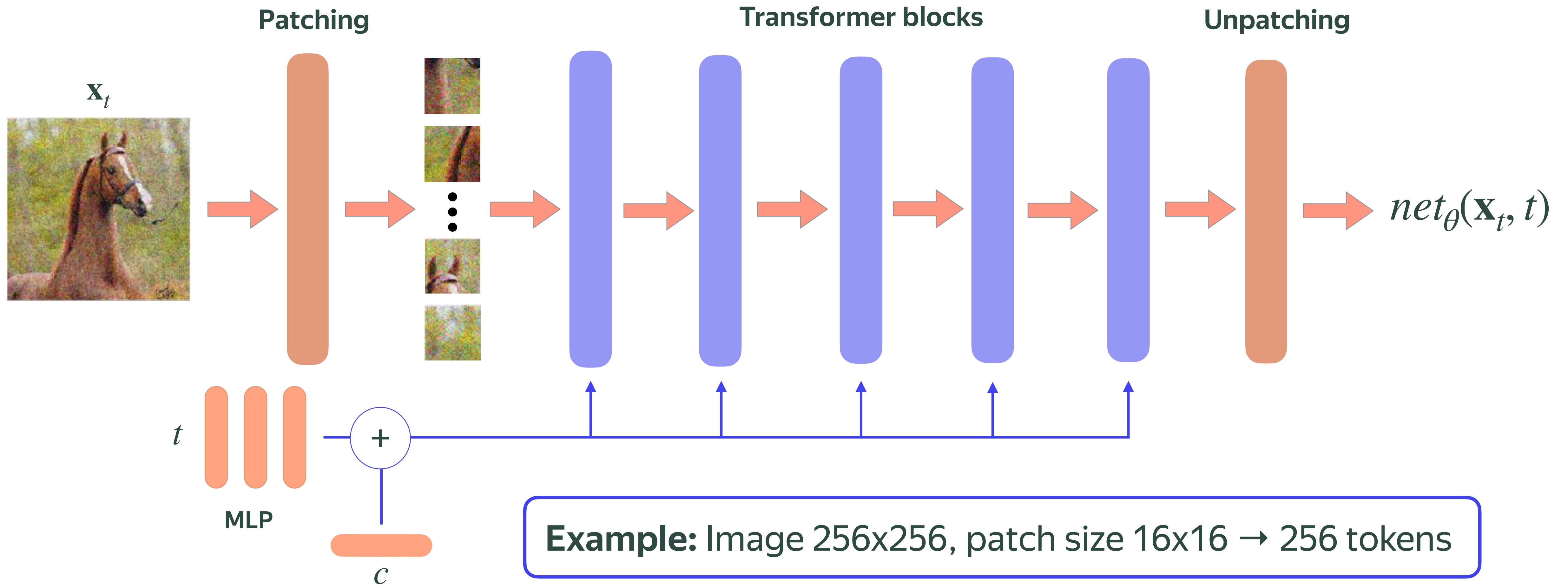
Conditional trick

$$\mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[w(t) \cdot \|\mathbf{v}_{\theta}(\mathbf{x}_t, t) - \mathbf{v}(\mathbf{x}_t)\|_2^2 \right] \overset{\text{red arrow}}{=} \mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[w(t) \cdot \|\mathbf{v}_{\theta}(\mathbf{x}_t, t) - \mathbf{v}^{\text{cond}}(\mathbf{x}_t)\|_2^2 \right] + C$$

Diffusion UNet



Diffusion transformer



Sampling with PF-ODE

Score Matching

$$d\mathbf{x} = \textcolor{red}{f(t)}\mathbf{x} - \frac{1}{2}\textcolor{blue}{g^2(t)} s_{\theta}(\mathbf{x}, t)dt$$

$$\textcolor{red}{f(t)} = \frac{d}{dt} \log \alpha_t = \frac{\dot{\alpha}_t}{\alpha_t}, \quad \textcolor{blue}{g^2(t)} = 2\sigma_t \dot{\sigma}_t - 2f(t)\sigma_t^2$$



$$d\mathbf{x} = \frac{\dot{\alpha}_t}{\alpha_t} \mathbf{x} - \sigma_t \left(\dot{\sigma}_t - \frac{\dot{\alpha}_t}{\alpha_t} \sigma_t \right) s_{\theta}(\mathbf{x}, t) dt = \left\{ s_{\theta}(\mathbf{x}, t) = -\frac{\epsilon_{\theta}(\mathbf{x}, t)}{\sigma_t} \right\} = \frac{\dot{\alpha}_t}{\alpha_t} \mathbf{x} + \left(\dot{\sigma}_t - \frac{\dot{\alpha}_t}{\alpha_t} \sigma_t \right) \epsilon_{\theta}(\mathbf{x}, t) dt$$

Sampling with PF-ODE

Score Matching

Solve it!
$$d\mathbf{x} = \frac{\dot{\alpha}_t}{\alpha_t} \mathbf{x} + \left(\dot{\sigma}_t - \frac{\dot{\alpha}_t}{\alpha_t} \sigma_t \right) \epsilon_{\theta}(\mathbf{x}, t) dt$$

Euler

DDIM

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \underbrace{\Delta t \left(\frac{\dot{\alpha}_t}{\alpha_t} \mathbf{x}_t + \left(\dot{\sigma}_t - \frac{\dot{\alpha}_t}{\alpha_t} \sigma_t \right) \epsilon_{\theta}(\mathbf{x}_t, t) \right)}_{\text{Linear part}} \longrightarrow \mathbf{x}_{t-\Delta t} = \alpha_{t-\Delta t} \left(\frac{\mathbf{x}_t - \sigma_t \epsilon_{\theta}(\mathbf{x}_t, t)}{\alpha_t} \right) + \sigma_{t-\Delta t} \epsilon_{\theta}(\mathbf{x}_t, t)$$

Linear part

$$\dot{\alpha}_t = \frac{\alpha_t - \alpha_{t-\Delta t}}{\Delta t}, \quad \dot{\sigma}_t = \frac{\sigma_t - \sigma_{t-\Delta t}}{\Delta t}$$

DDIM == DPM-1 solver

Sampling | Flow matching

Solve ODE $d\mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)dt$

Euler (1st-order)

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \mathbf{v}_\theta(\mathbf{x}_t, t)$$

Heun (2nd-order)

$$\begin{aligned}\tilde{\mathbf{x}}_{t-\Delta t} &= \mathbf{x}_t - \Delta t \cdot \mathbf{v}_\theta(\mathbf{x}_t, t) \\ \mathbf{x}_{t-\Delta t} &= \mathbf{x}_t - \frac{\Delta t}{2} \left(\mathbf{v}_\theta(\mathbf{x}_t, t) + \mathbf{v}_\theta(\tilde{\mathbf{x}}_{t-\Delta t}, t - \Delta t) \right)\end{aligned}$$

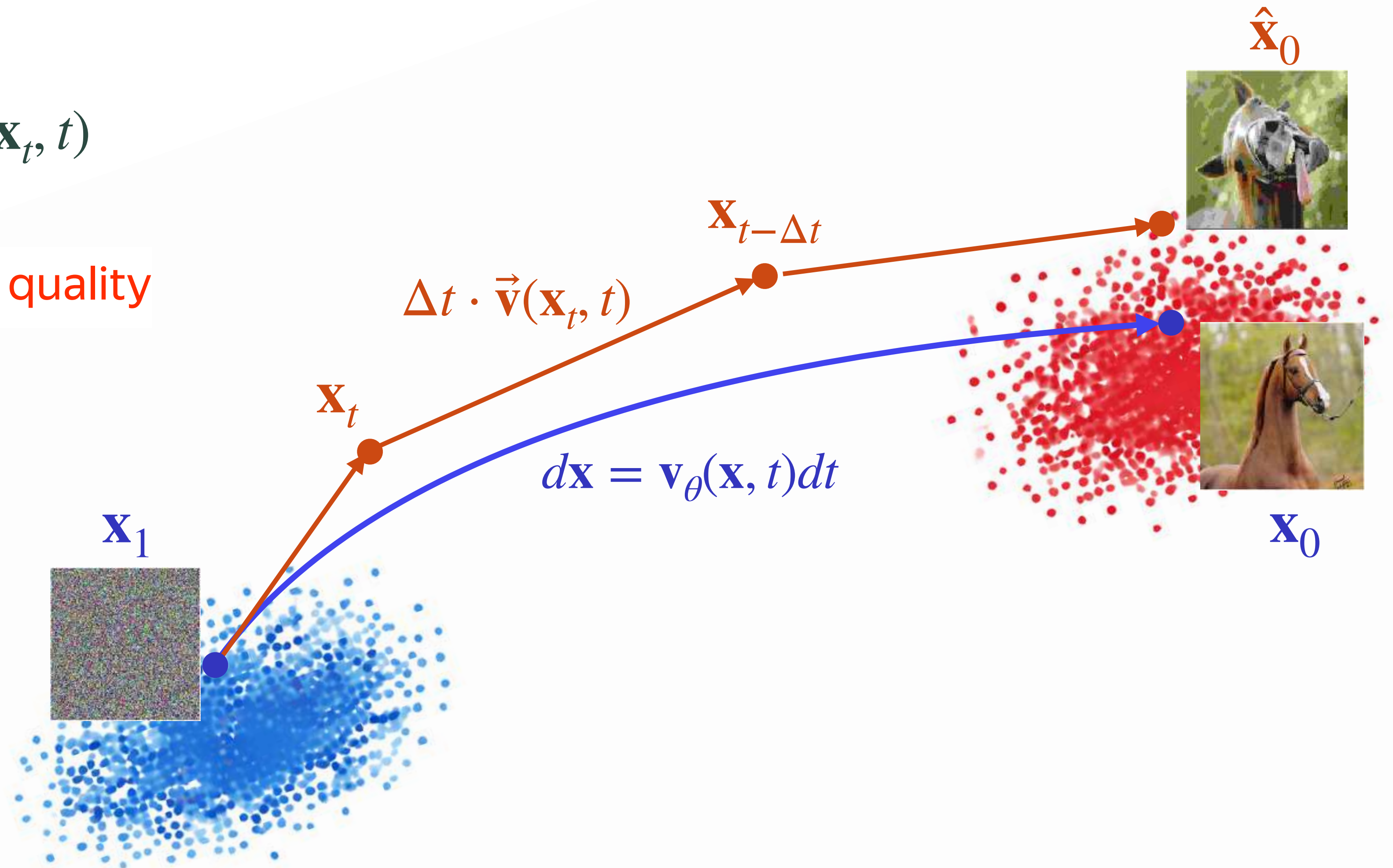
Euler / Heun == DPM-1 / DPM-2 solvers

* for flow matching

Speed-quality trade-off

Euler: $\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \mathbf{v}_\theta(\mathbf{x}_t, t)$

Large $\Delta t \rightarrow$ **few steps** & **lower quality**

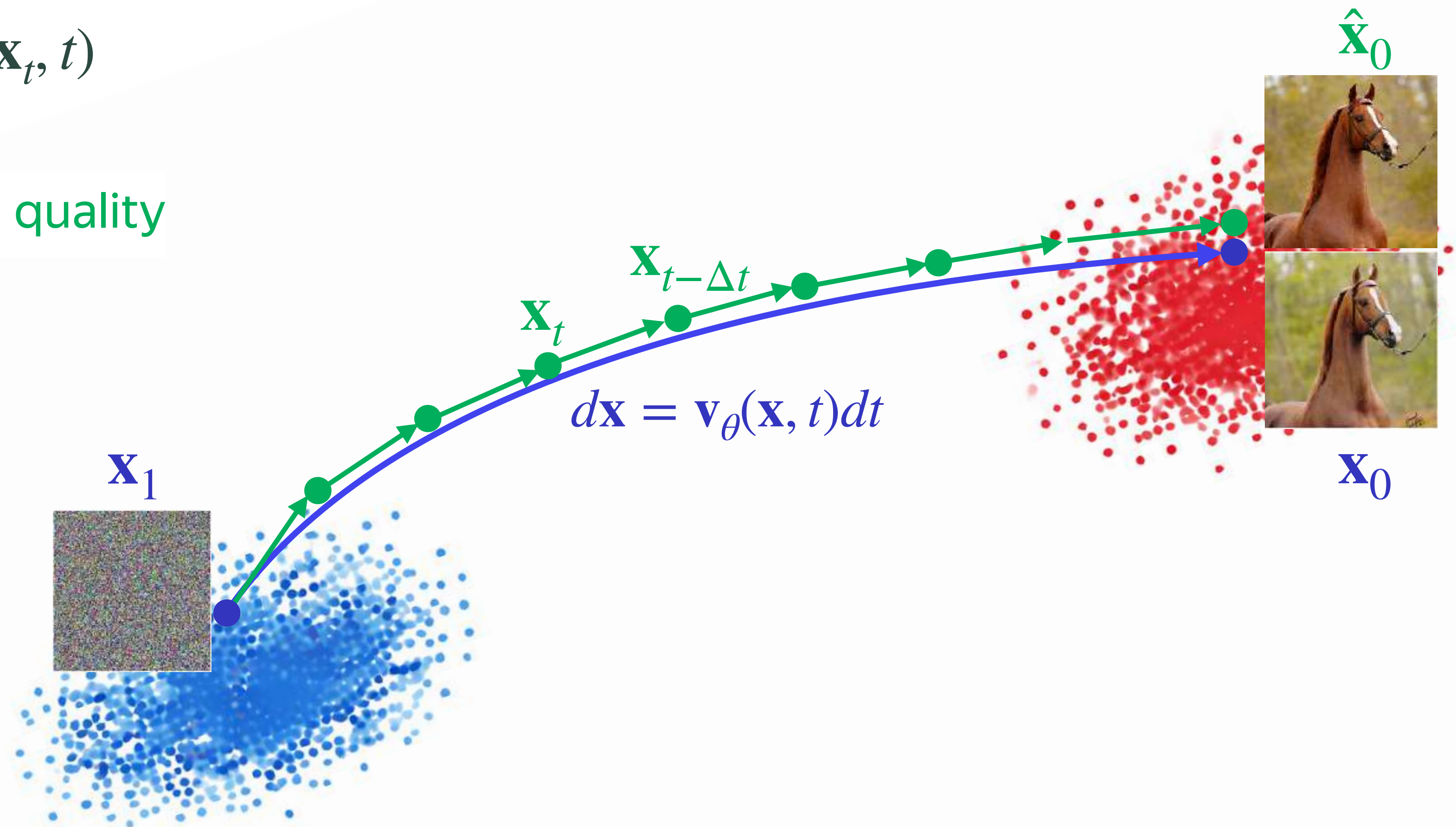


Speed-quality trade-off

Euler: $\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \mathbf{v}_\theta(\mathbf{x}_t, t)$

Small $\Delta t \rightarrow$ many steps & high quality

Practical note
2nd-order solvers can do better but
Euler is a default choice in practice



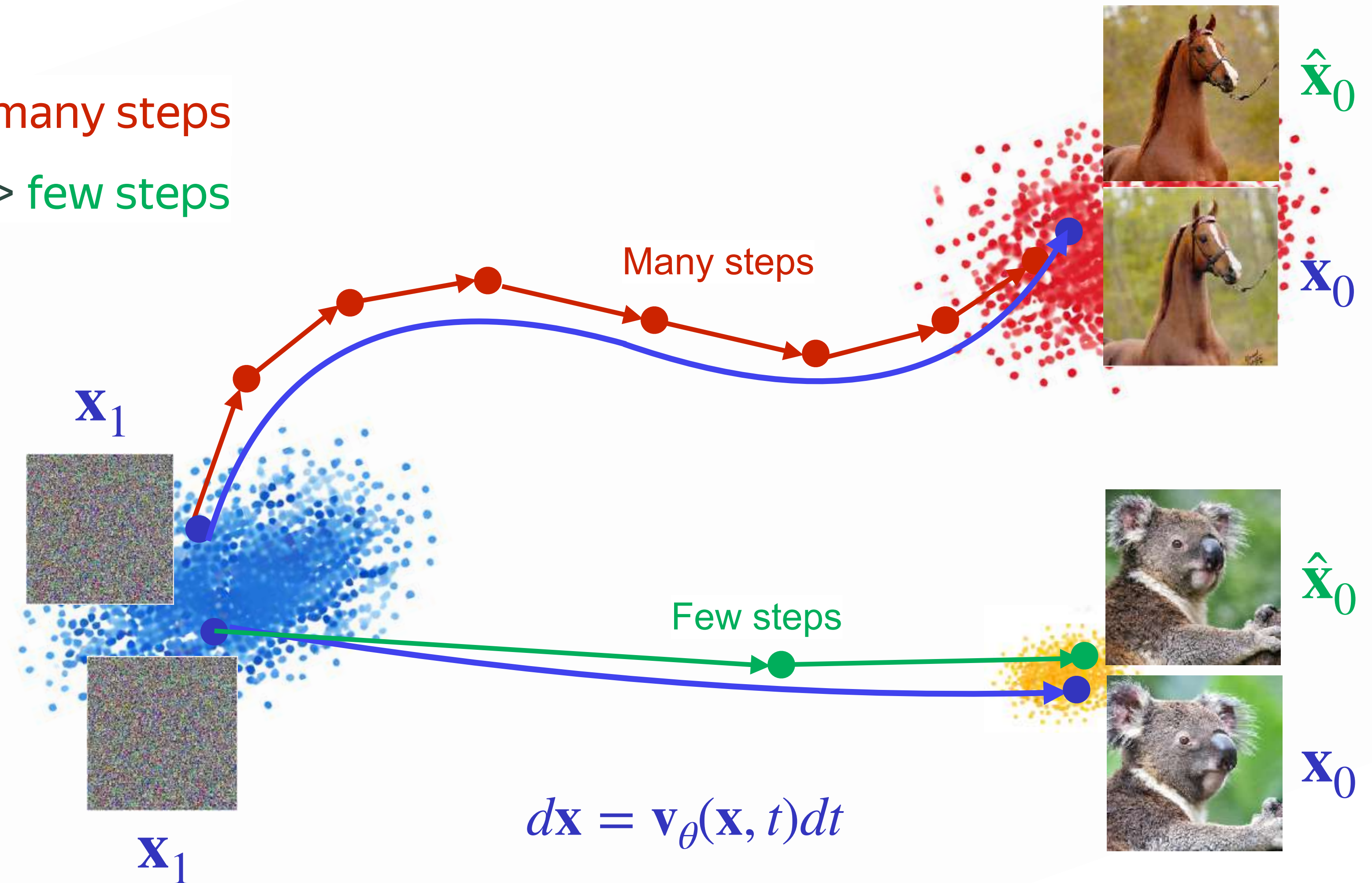
Effect of the trajectory curvature

Curvy trajectories $\rightarrow$ small $\Delta t \rightarrow$ many steps

Straight trajectories $\rightarrow$ large $\Delta t \rightarrow$ few steps

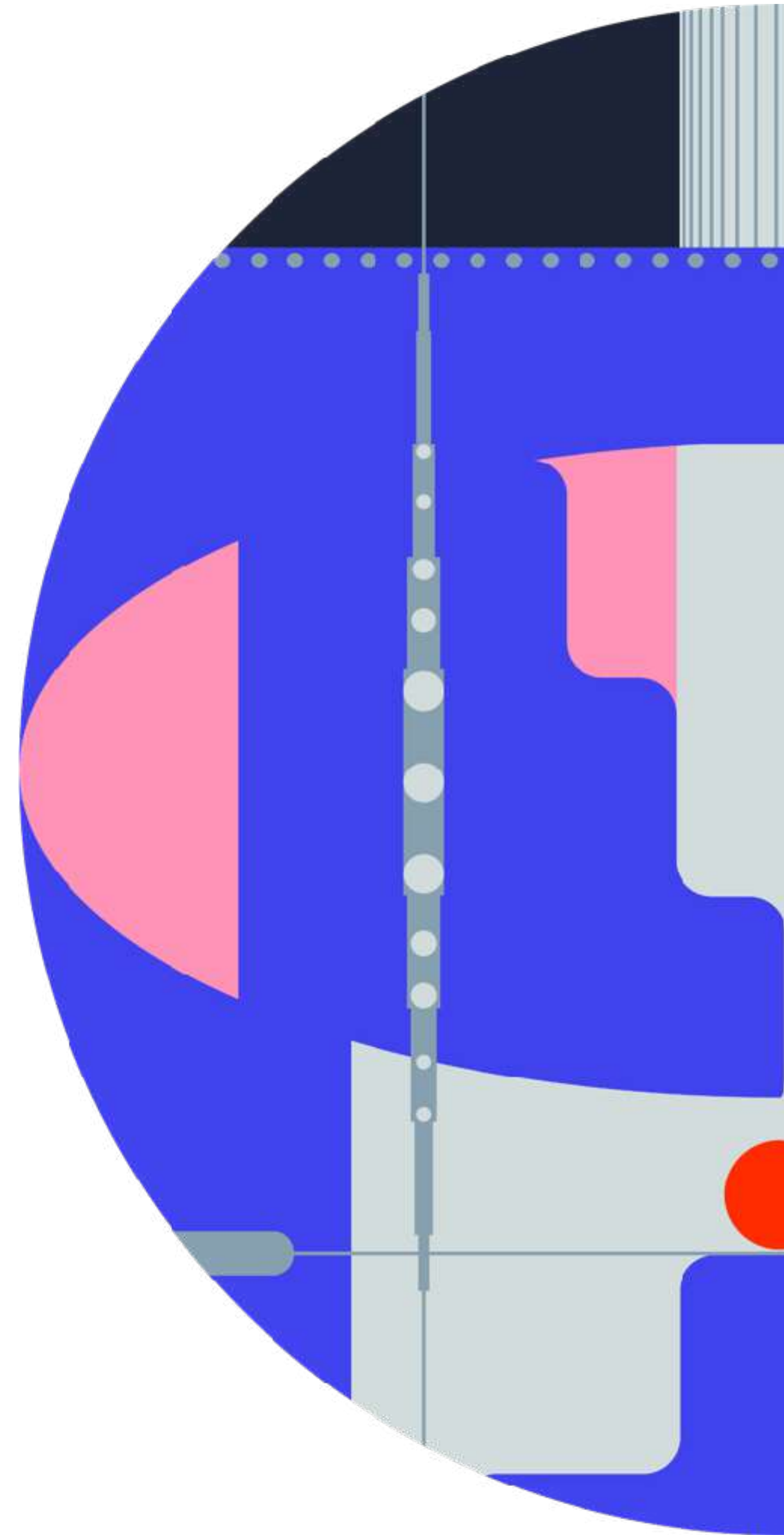
Practical note

The most effective way to get straighter trajectories — stronger condition, e.g., text or another image



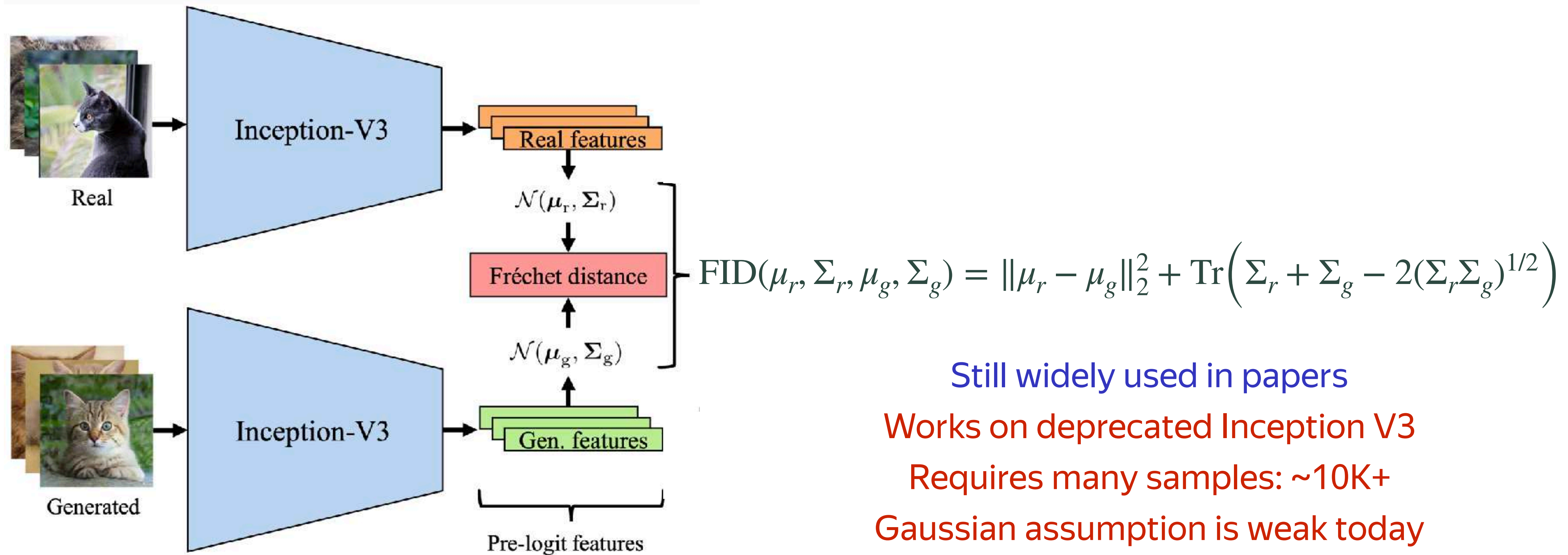
How to assess generative models?

*In academic scenarios



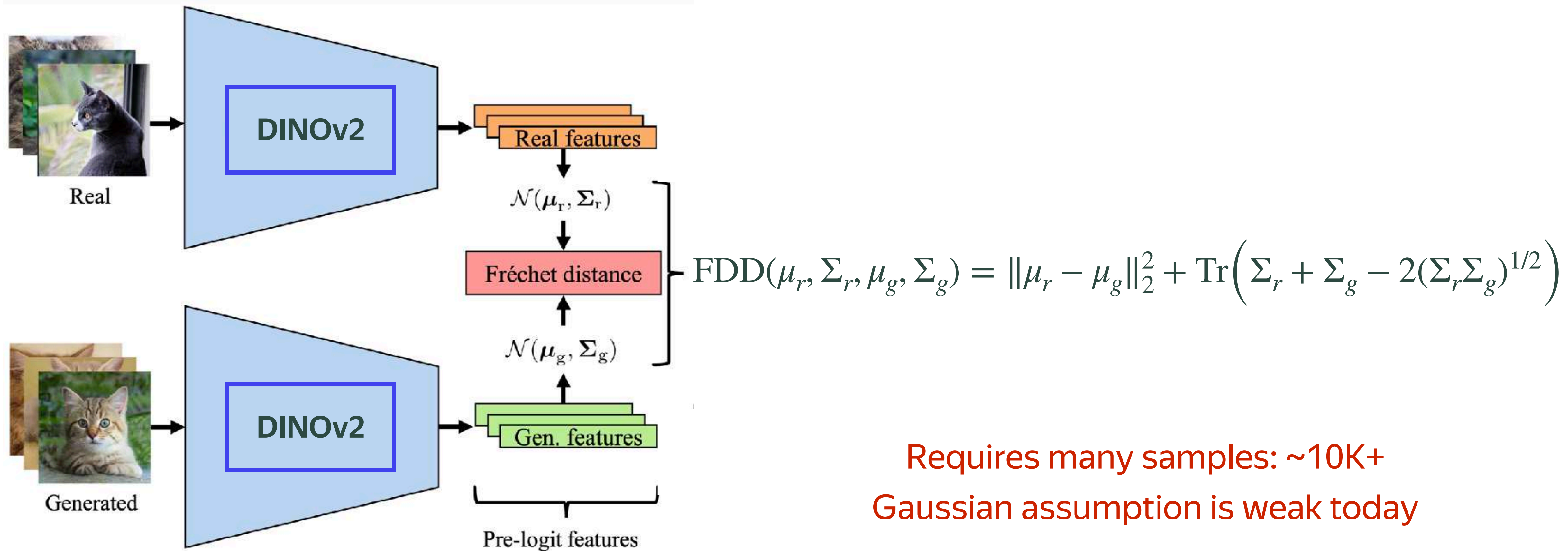
Fréchet Inception Distance (FID)

2-Wasserstein distance between real and fake **Gaussian** distributions in the **Inception V3** feature space



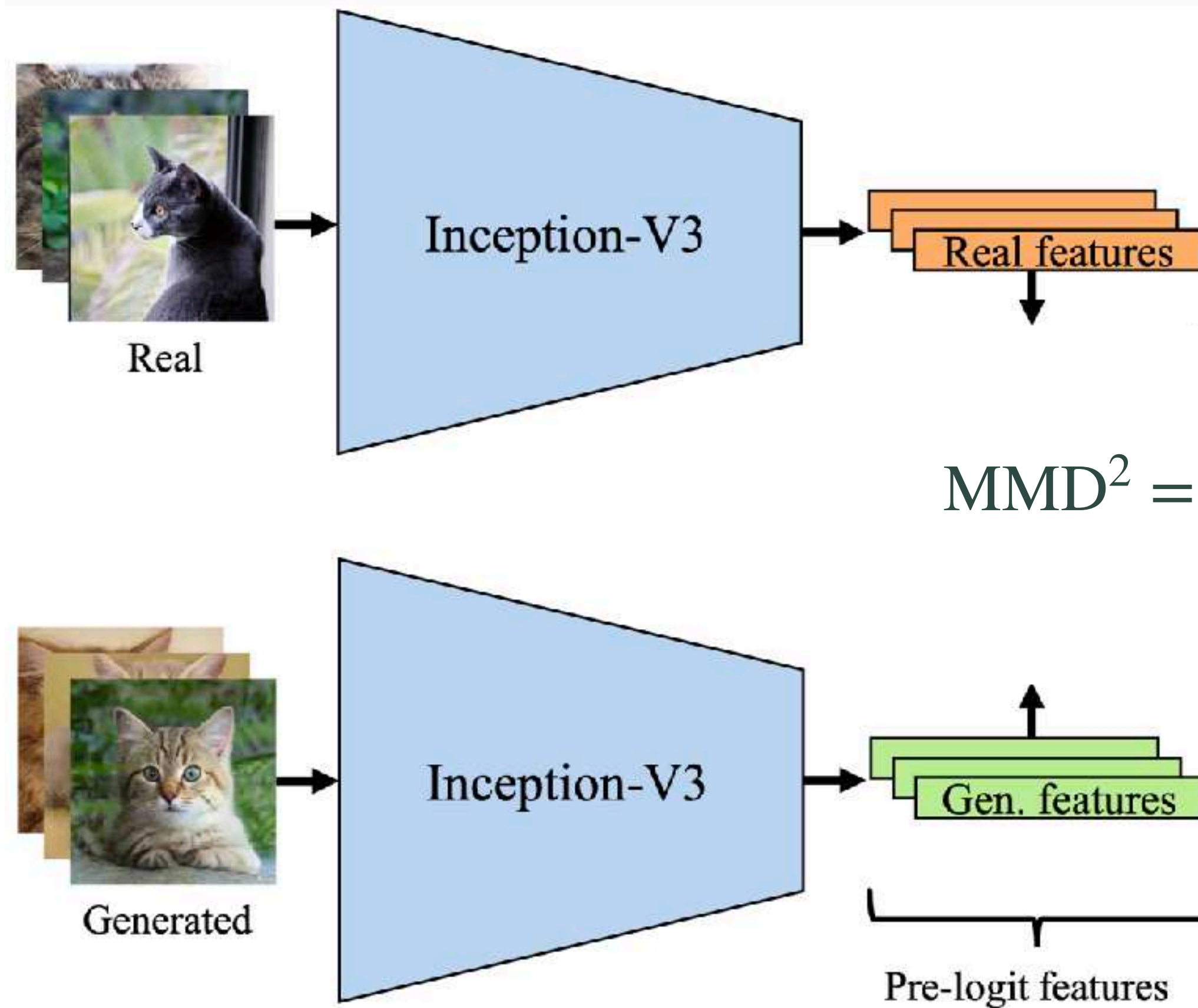
Fréchet Distance DINOv2 (FDD)

2-Wasserstein distance between real and fake **Gaussian** distributions in the **DINOv2** feature space



Kernel Inception Distance (KID)

Maximum Mean Discrepancy (MMD) between real and fake distributions in the **Inception V3** feature space



Idea: pairwise distances using $k(x, y)$

Polynomial kernel: $k(x, y) = \left(\frac{1}{d} x^\top y + 1 \right)^3$

$$\text{MMD}^2 = \frac{1}{n(n-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j)$$

OK for much fewer samples: ~1K
No gaussian assumption anymore
Works on deprecated Inception V3

Takeaways

FID — **legacy in papers**. OK for academic datasets. **No way in s.o.t.a. text-to-image**

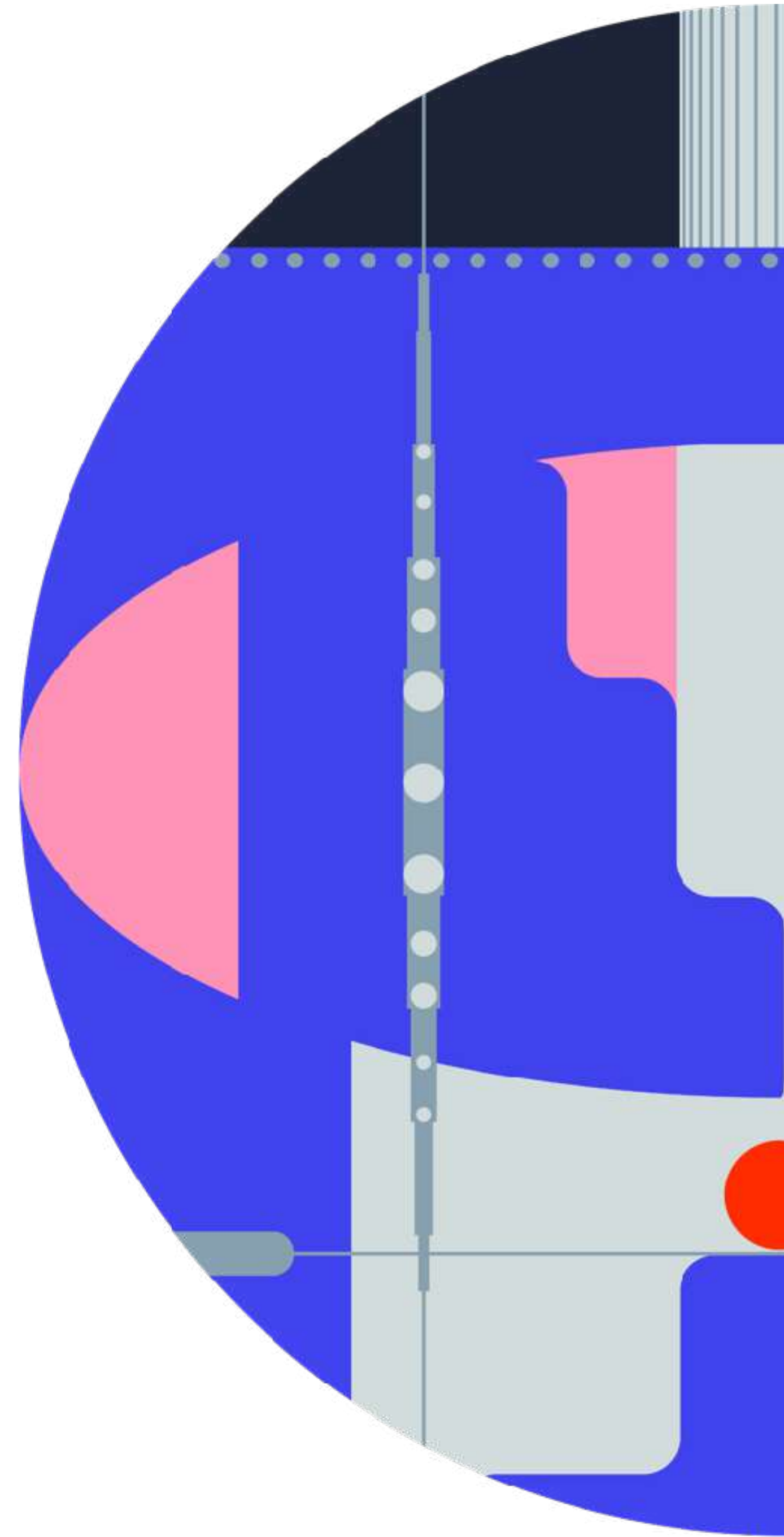
KID — used instead FID when few real/fake samples are available for evaluation

FDD — more recent and reliable FID variant but still not good enough for real-world cases

There are other metrics for academic cases like Precision and Recall

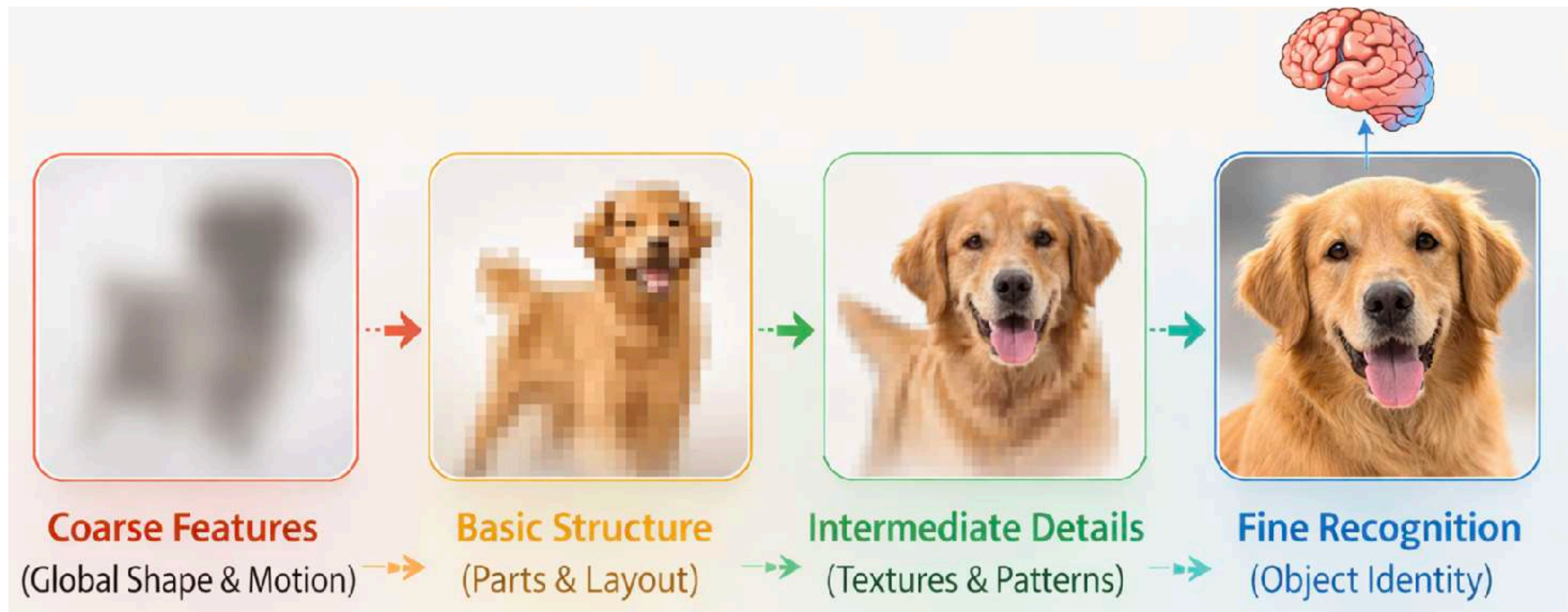
Text-to-image FID/KID variants, e.g., **CMMD**, seem reasonable but not quite popular

Why diffusion for visual generation?



How do people process image?

Human vision processes images hierarchically: from **global structure** to **fine details**



How do people generate images?

Pretty much similar...



FLUX.2

Diffusion also generates images in coarse-to-fine manner

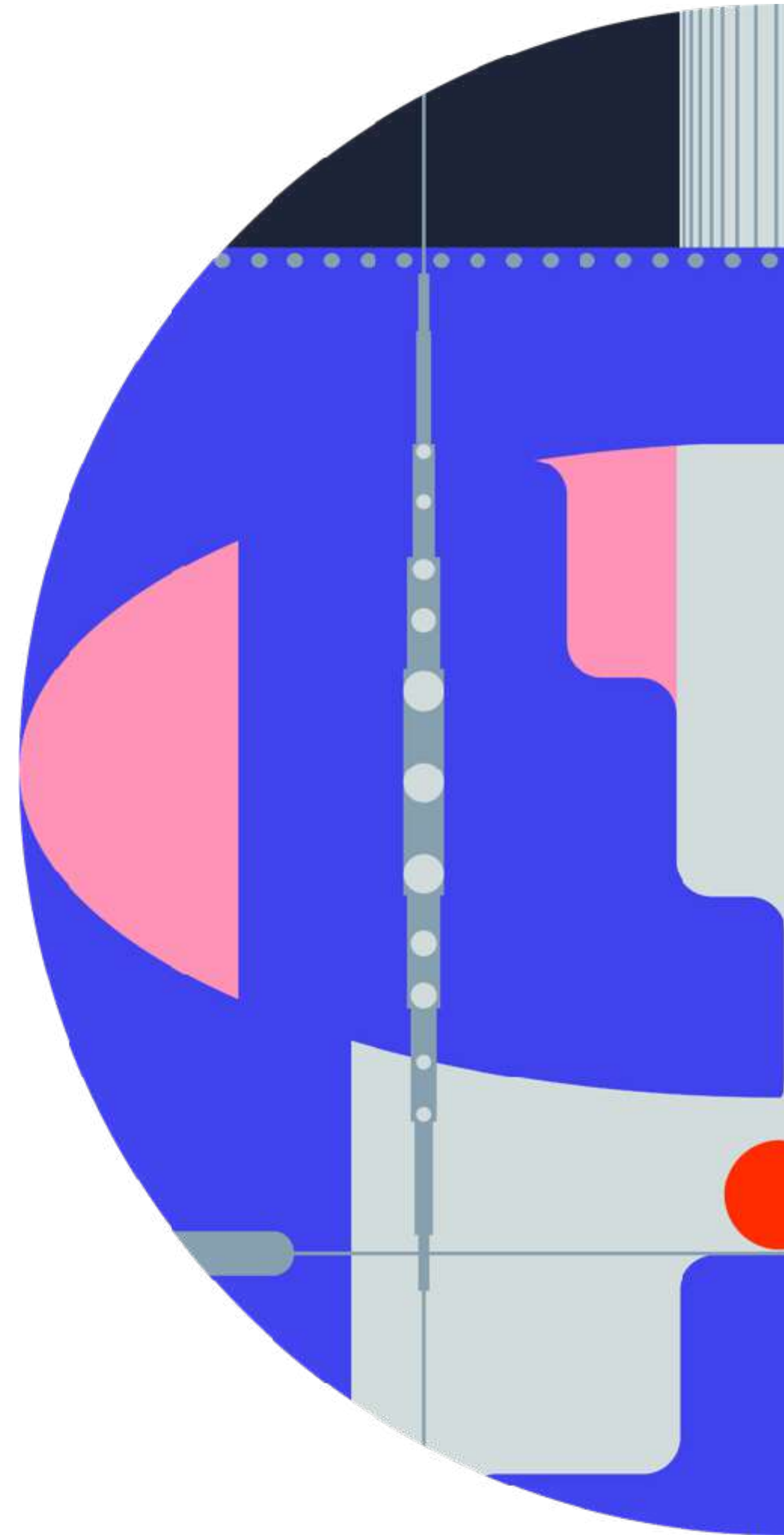


Layout and global shapes

Main object details

Fine-grained details
Textures, text, hair, grass

How to design your diffusion model?



How to design your diffusion model?

Design choices

1. Choice of $p(t)$ and $w(t)$
2. Network prediction $net_{\theta}(\mathbf{x}_t, t)$ for $\mathbf{v}_{\theta}$
3. Diffusion space (pixel vs latent)
4. Representation alignment

Loss related

Design related

Forward process: $\mathbf{x}_t = (1 - t) \cdot \mathbf{x} + t \cdot \epsilon$

v-loss: $\mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[w(t) \cdot \|v_{\theta}(\mathbf{x}_t, t) - \mathbf{v}\|_2^2 \right]$

Loss-weighting vs Timestep distribution

Both $p(t)$ and $w(t)$ are equivalent for loss weighting

$$\mathbb{E}_{t \sim p(t)} \mathbb{E}_{\mathbf{x}_t \sim p(\mathbf{x}_t)} \left[\cancel{w(t)} \cdot \|v_\theta(\mathbf{x}_t, t) - \mathbf{v}\|_2^2 \right]$$

Always $w(t) \equiv 1$ and focus on $p(t)$

$p(t)$ gives more weight by sampling more frequently

$p(t)$ spends less compute on non-effective t

What is a good choice for $p(t)$?

Why intermediate timesteps matter

Intermediate timesteps are responsible for main content generation



Relatively easy

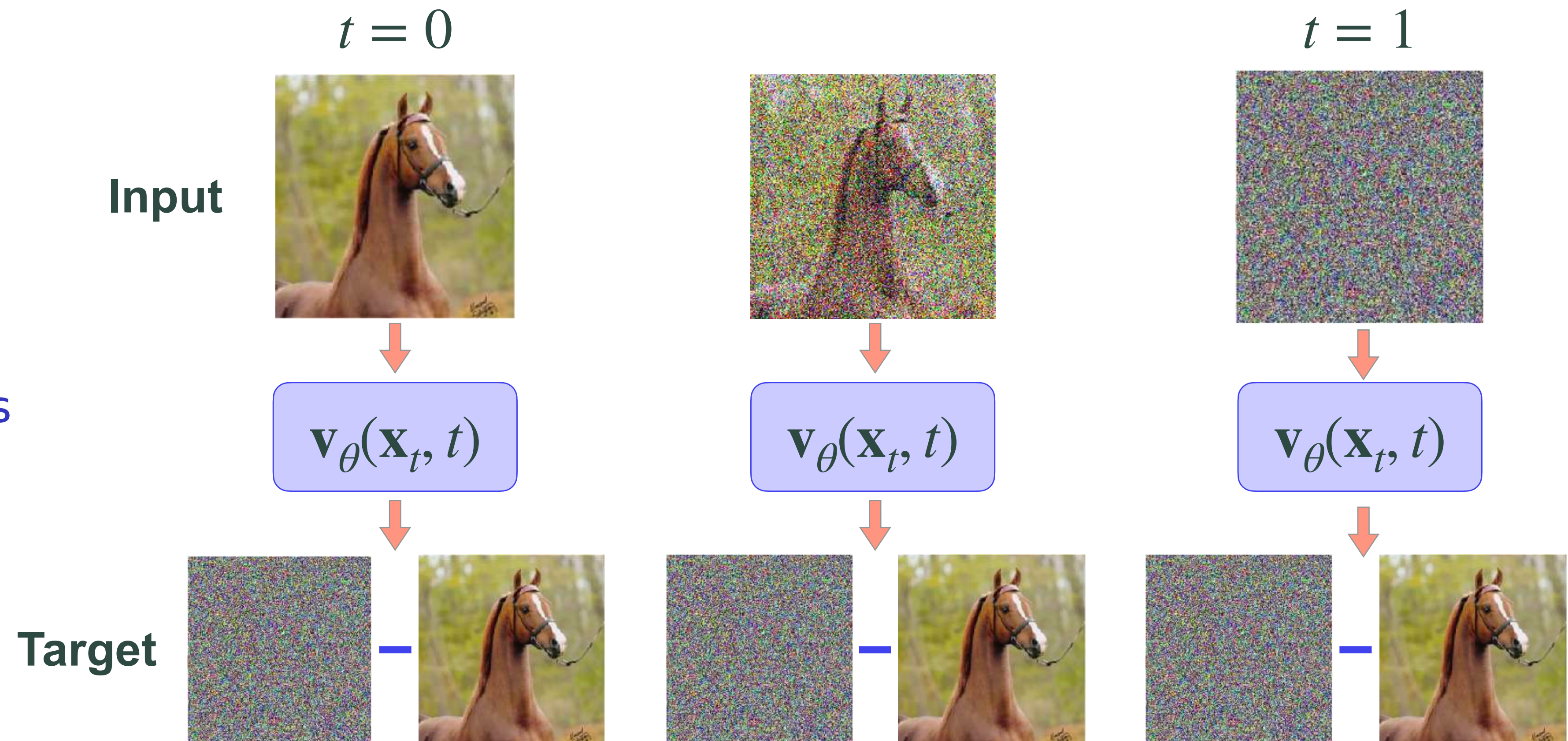
Difficult and important

Relatively easy

What do we make diffusion to predict?

$$\mathbb{E} \|\mathbf{v}_{\theta}(\mathbf{x}_t, t) - \mathbf{v}\|_2^2$$

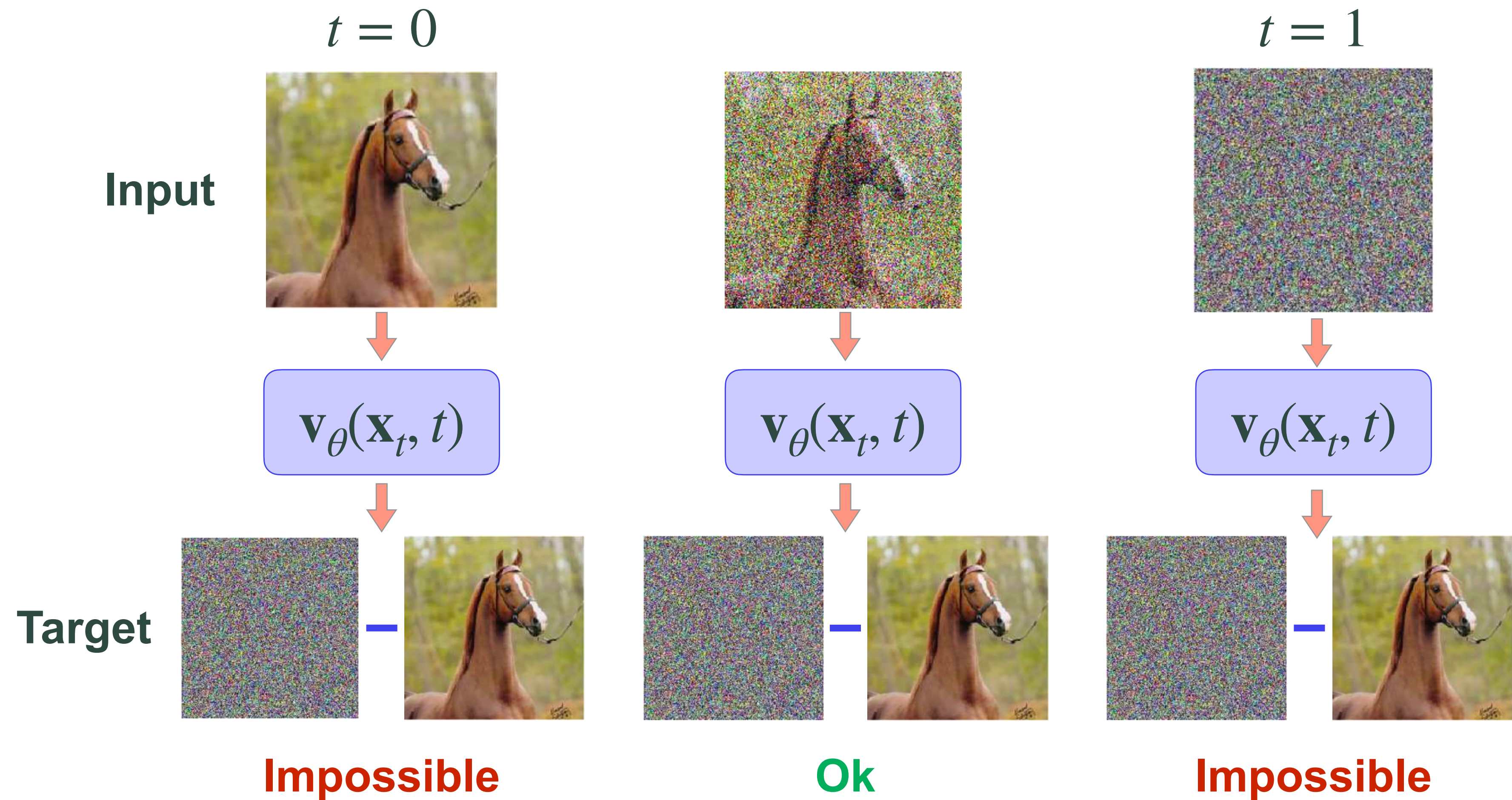
What are optimal predictions
at $t = 0$ and $t = 1$?



What do we make diffusion to predict?

$$\mathbb{E} \|\mathbf{v}_{\theta}(\mathbf{x}_t, t) - \mathbf{v}\|_2^2$$

Intermediate timesteps are more reasonable to learn



Effective timestep distribution

Let's make $p(t)$ focus on intermediate timesteps

Logit-normal distribution

$$p_{\text{ln}}(t; \mu, \sigma) = \frac{1}{\sigma \sqrt{2\pi} t(1-t)} \exp\left(-\frac{(\text{logit}(t) - \mu)^2}{2\sigma^2}\right)$$

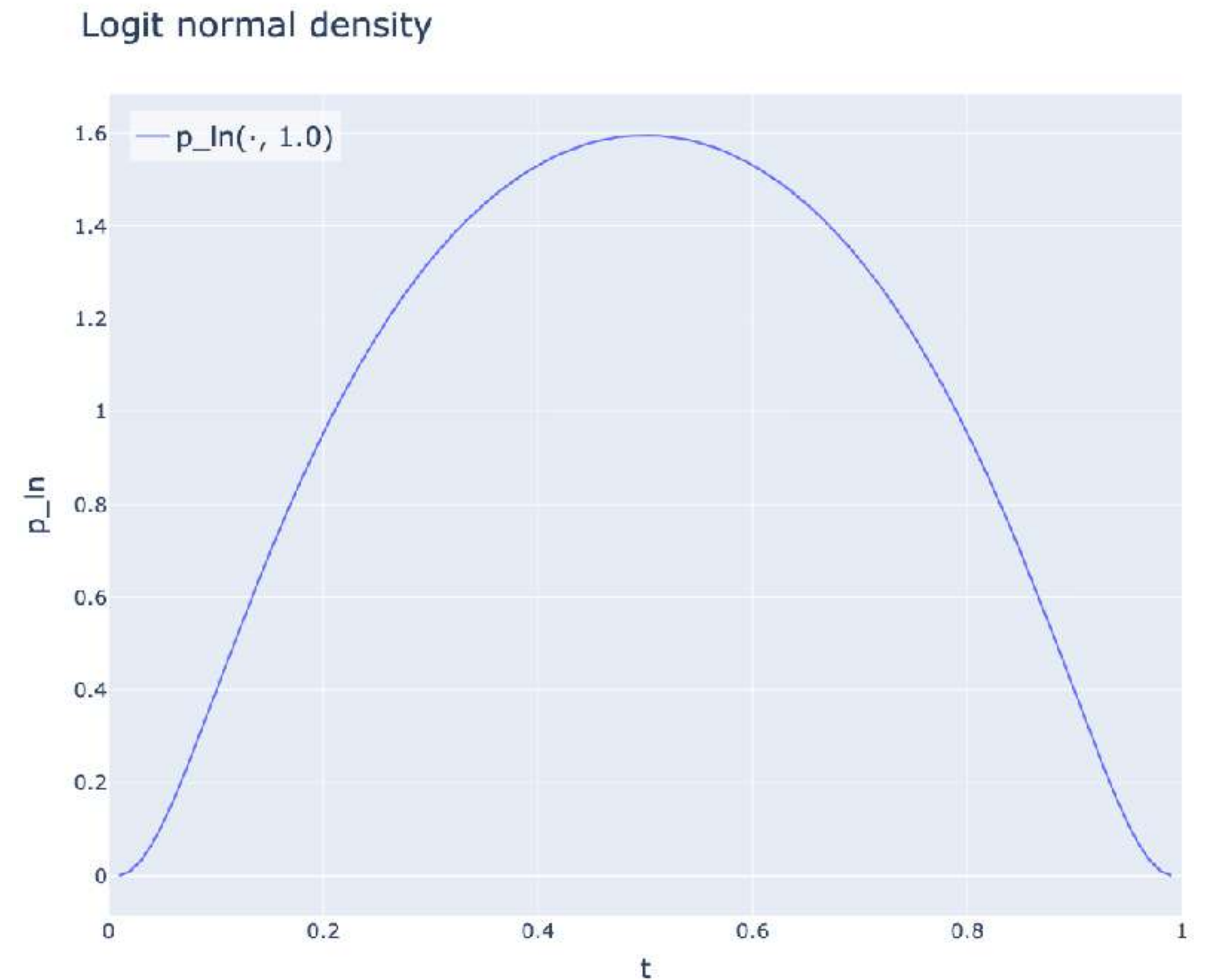
$$\text{logit}(t) = \log\left(\frac{t}{1-t}\right) \quad \mu = \log \alpha, \quad \sigma = 1$$

Effective timestep distribution

Let's make $p(t)$ focus on intermediate timesteps

Logit-normal distribution in practice

$$p_{\ln}(t; \alpha) = \frac{1}{\sqrt{2\pi} t(1-t)} \exp\left(-\frac{(\text{logit}(t) - \log \alpha)^2}{2}\right)$$



Effective timestep distribution

Let's make $p(t)$ focus on intermediate timesteps

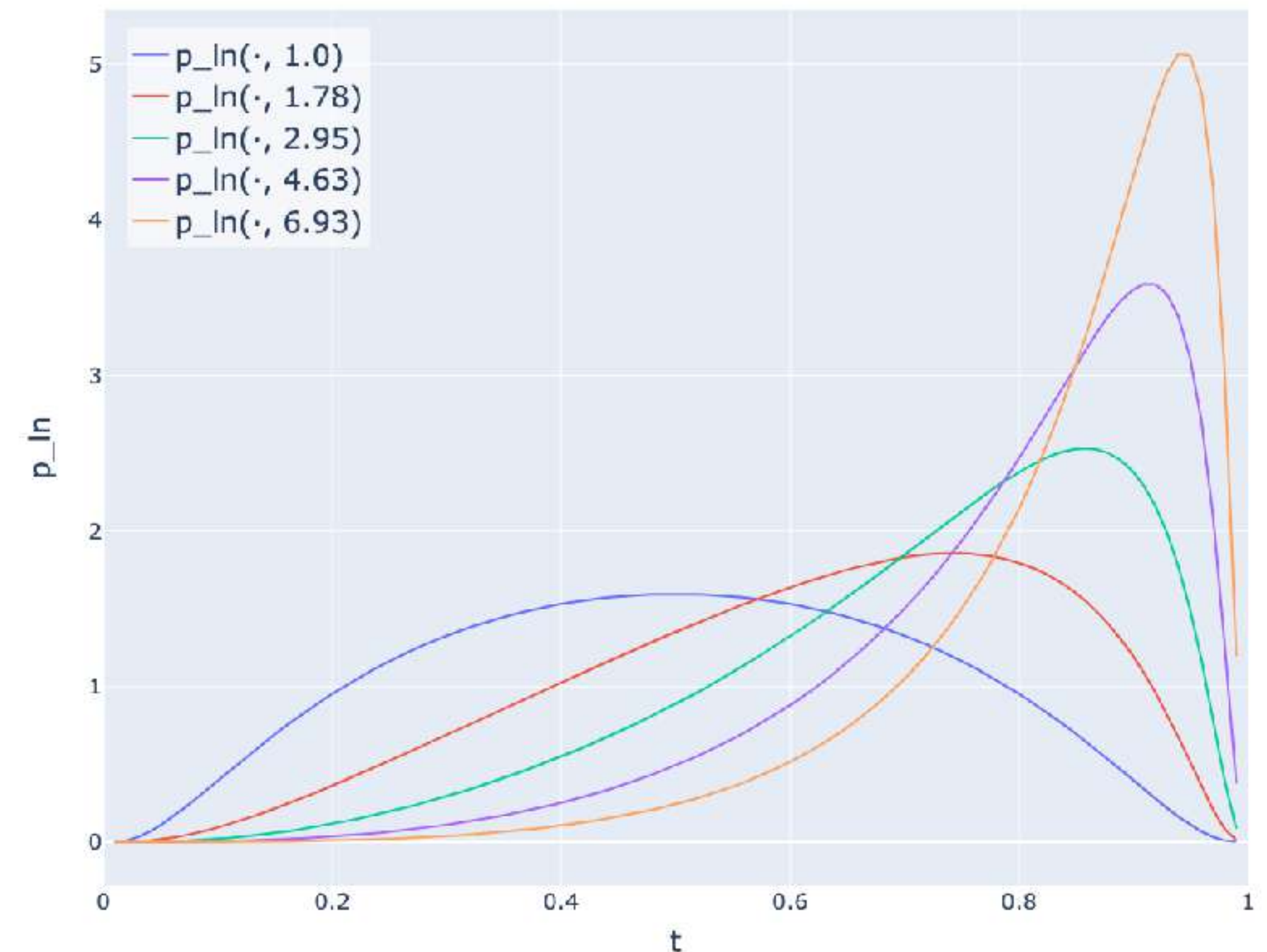
Logit-normal distribution in practice

$$p_{\ln}(t; \alpha) = \frac{1}{\sqrt{2\pi} t(1-t)} \exp\left(-\frac{(\text{logit}(t) - \log \alpha)^2}{2}\right)$$

Often, it is beneficial to **shift** it to high noise levels

$$\alpha > 1$$

Shifted logit normal density



Why shifting $p(t)$?

Goal: understand what happens with $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\epsilon$ at **different image resolutions**

Why shifting $p(t)$?

Proxy for a real image $\mathbf{x}_0$

Let's consider a dummy random variable $Y(t) = (1 - t)c + t\epsilon$, where $c \in \mathbb{R}$, $\epsilon \sim \mathcal{N}(0,1)$

$$\mathbb{E} [Y(t)] = \mathbb{E} [(1 - t)c + t\epsilon] = (1 - t)c \rightarrow \text{given } N \text{ observations of } Y(t): \hat{c} = \frac{1}{1 - t} \frac{1}{N} \sum_i^N y_i$$

$$\text{Var} [Y(t)] = \text{Var}[(1 - t)c + t\epsilon] = t^2 \rightarrow \text{Var} [\hat{c}] = \frac{1}{(1 - t)^2} \frac{1}{N^2} \sum_i^N \text{Var} [y_i] = \frac{t^2}{(1 - t)^2} \frac{1}{N}$$

Why shifting $p(t)$?

$$\text{Var} [Y(t)] = \frac{t^2}{(1-t)^2} \frac{1}{N} \rightarrow \sigma(t, N) = \frac{t}{1-t} \sqrt{\frac{1}{N}} \rightarrow \text{More samples } N \text{ reduce uncertainty as } \sqrt{\frac{1}{N}}$$

Same noise level t destroys data $\mathbf{x}_0$ less at higher resolution

$t = 0.25$

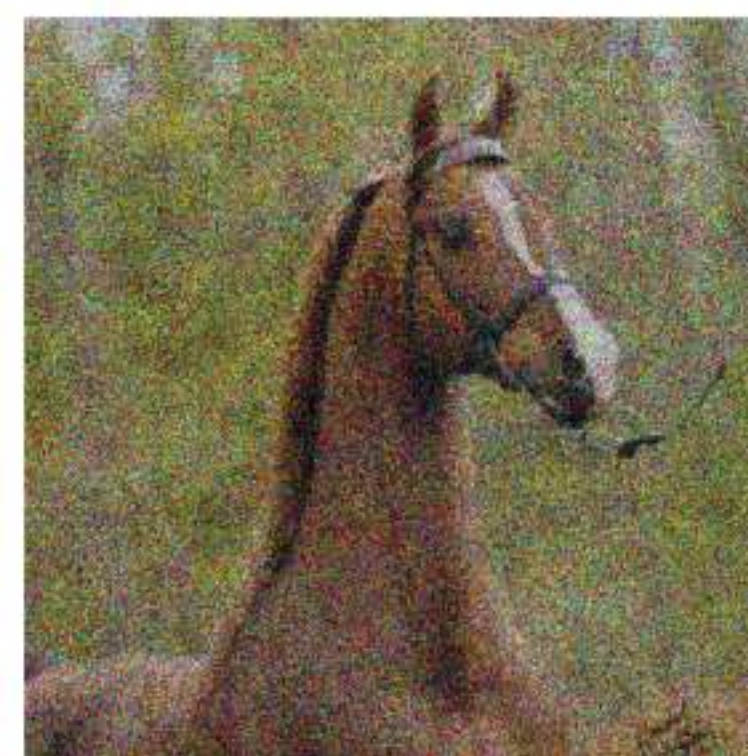
128x128



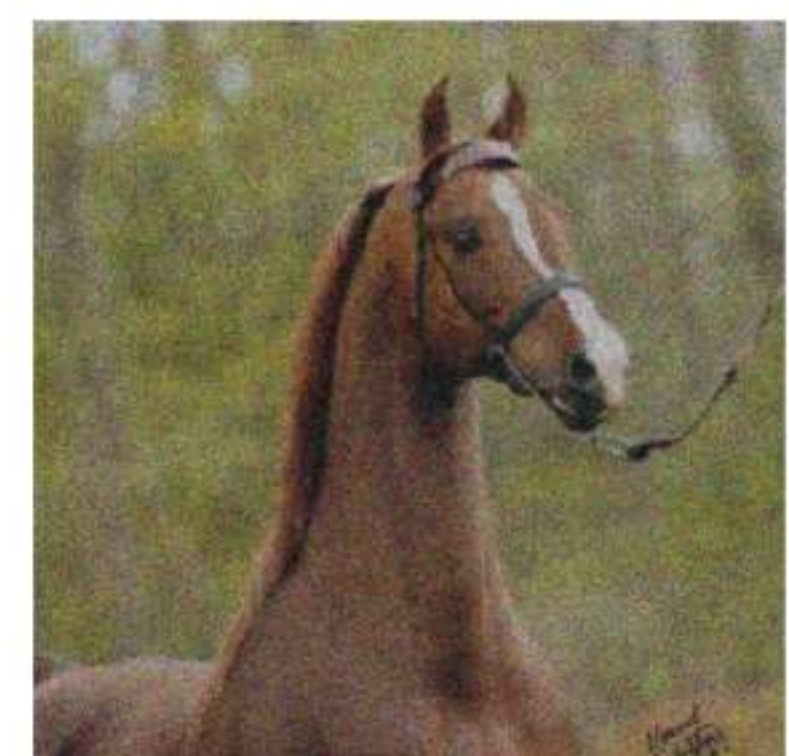
256x256



512x512



1024x1024

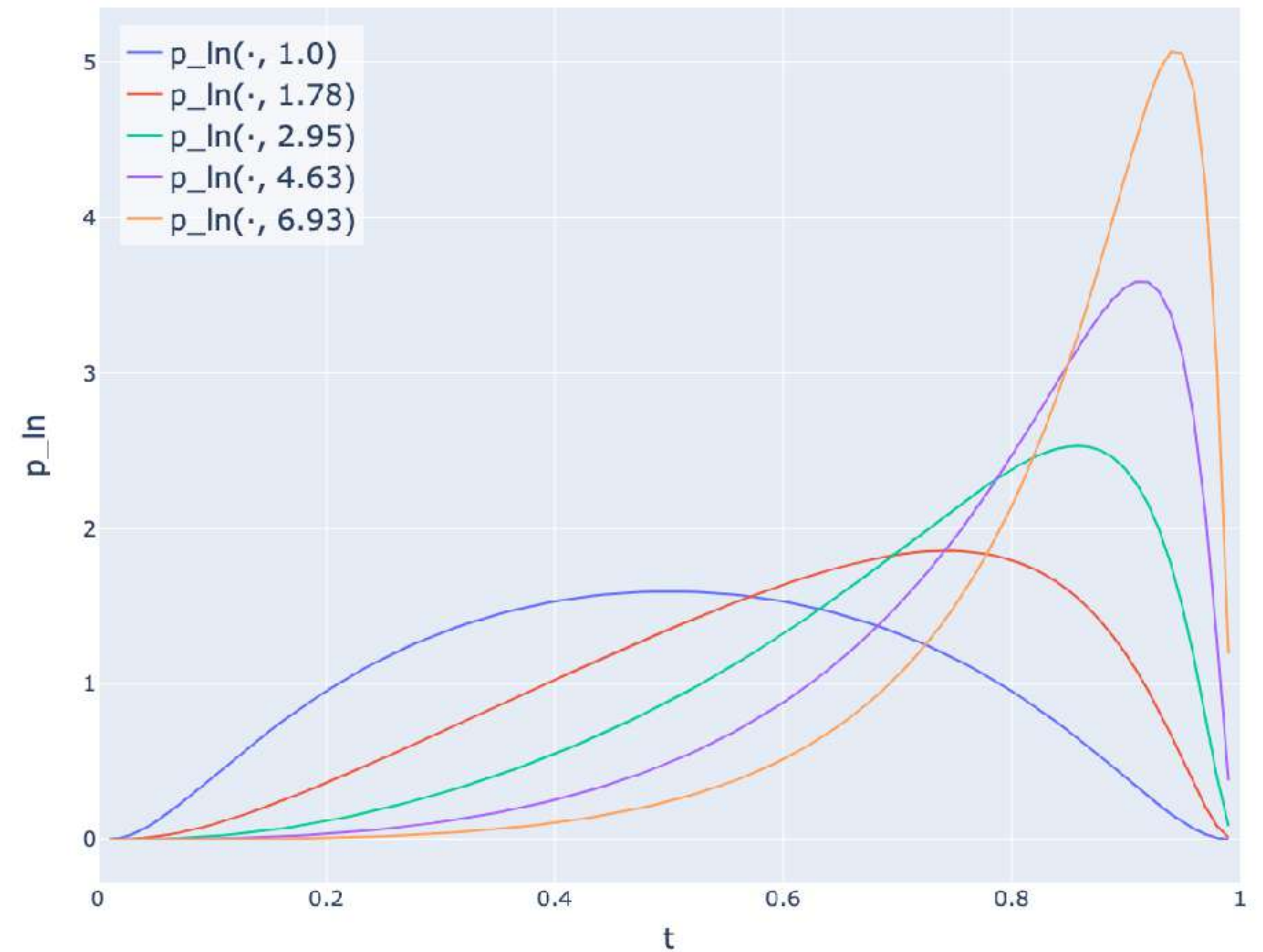


Takeaway

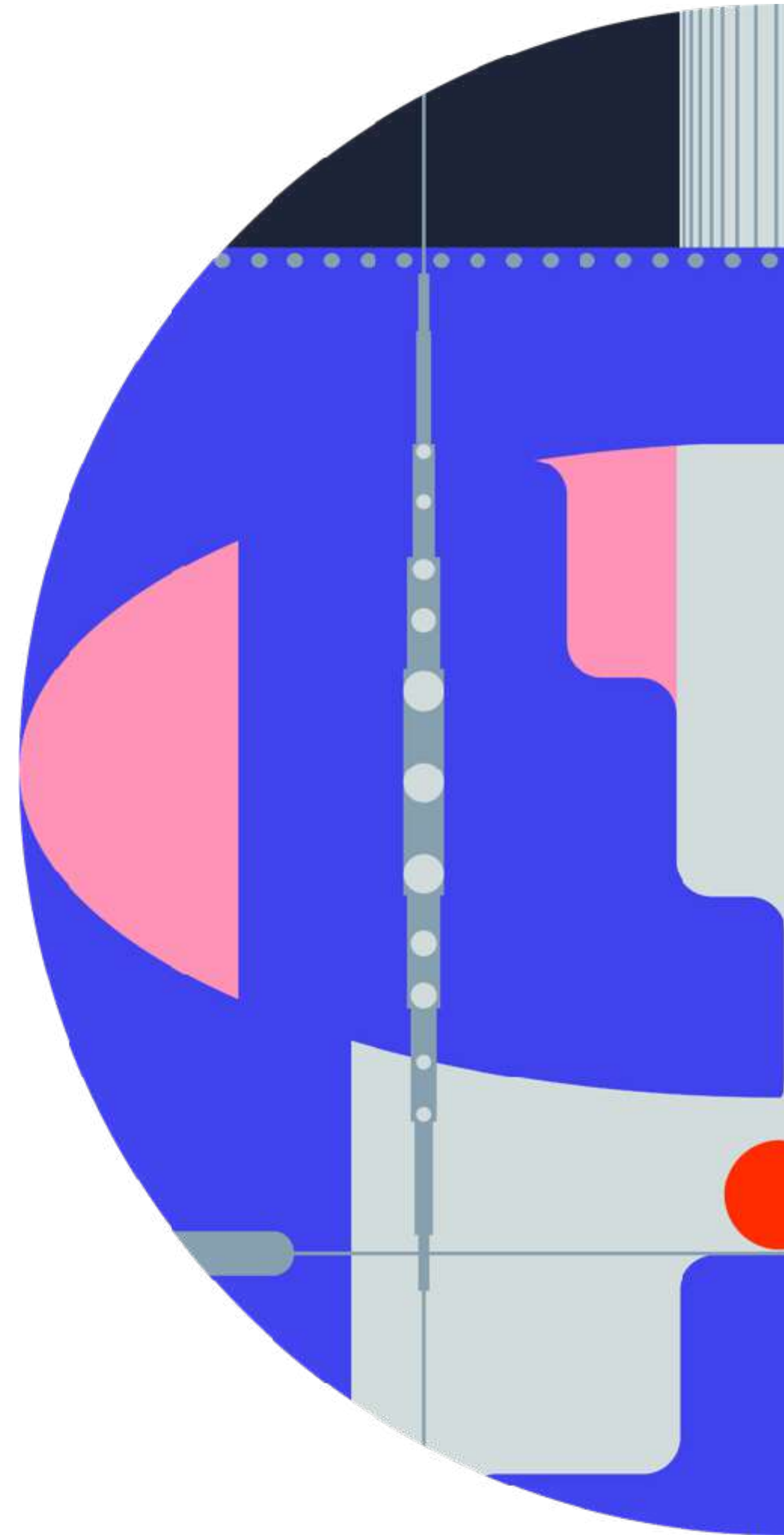
$$p_{\ln}(t; \alpha) = \frac{1}{\sqrt{2\pi} t(1-t)} \exp\left(-\frac{(\text{logit}(t) - \log \alpha)^2}{2}\right)$$

Higher resolution $\rightarrow$ larger shift α

Shifted logit normal density



Diffusion spaces



Network prediction in pixel space

Noise in ϵ , $\mathbf{v}$ make target full dimensional



Difficult for $net_{\theta}(\mathbf{x}_t, t)$ to learn it

$\mathbf{x}_0$ lies on lower dimensional subspace

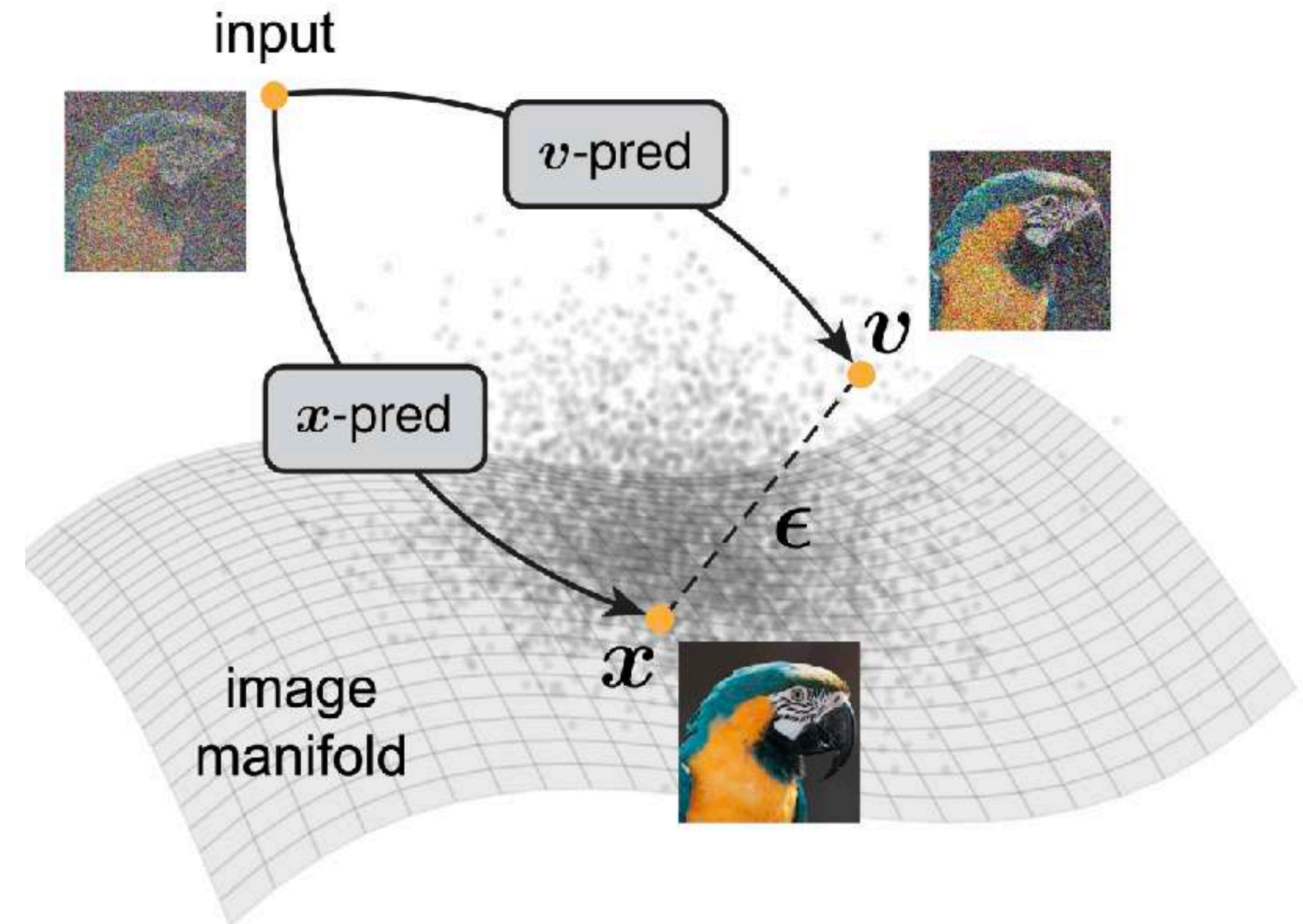


Easier for $net_{\theta}(\mathbf{x}_t, t)$ to learn it

Pixel-space diffusion $\rightarrow \mathbf{x}_0^{\theta} := net_{\theta}(\mathbf{x}_t, t)$

Manifold Assumption

Natural images lie on a low-dimensional manifold in the high-resolution pixel space



Is it really critical?

$\mathbf{x}_0$ -prediction is **must-have** at high resolution

Dataset	$\mathbf{x}_0$ -pred	ϵ -pred	$\mathbf{v}$ -pred
ImageNet 256×256	8.62	372.38	96.53
ImageNet 64×64	3.55	3.63	3.46

FID results for different pred types

Problem

Even $\mathbf{x}_0$ is relatively high dimensional



Pixel space models are not s.o.t.a. **yet**

What is the problem with pixel space?

Natural images contain much imperceptible noise and high-frequency details



Natural images remain relatively high dimensional

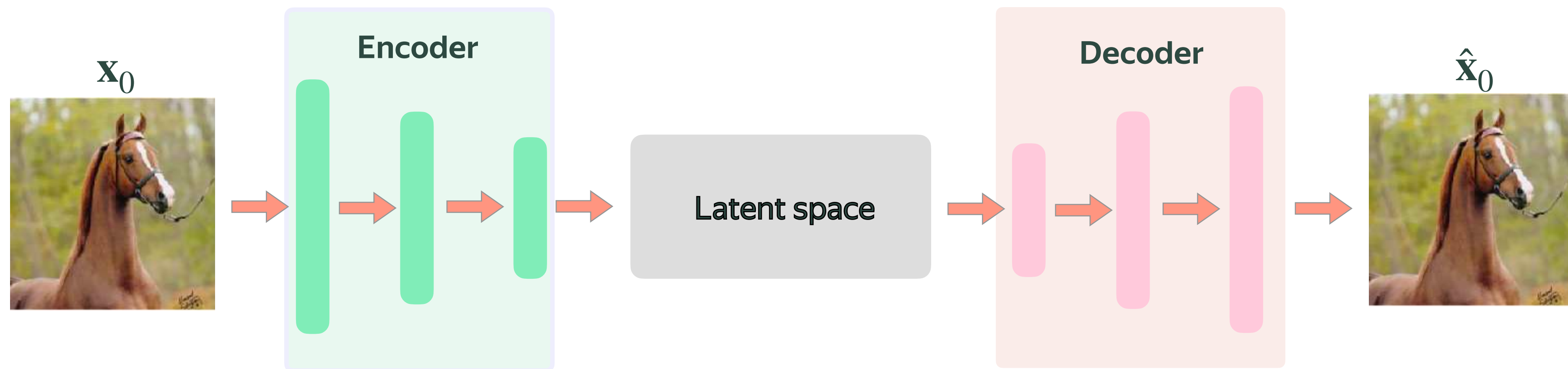


Diffusion models spend much capacity to fit this



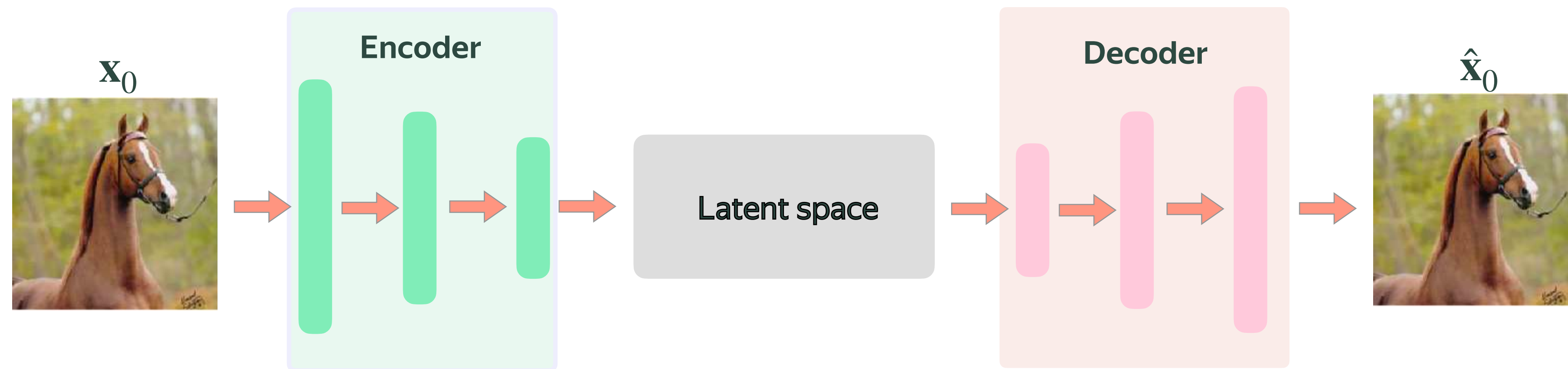
Latent diffusion models

Use VAE to map images into lower dimensional latent space



Latent diffusion models

The V in VAE is **vestigial**: in fact, it just AE with slight regularization



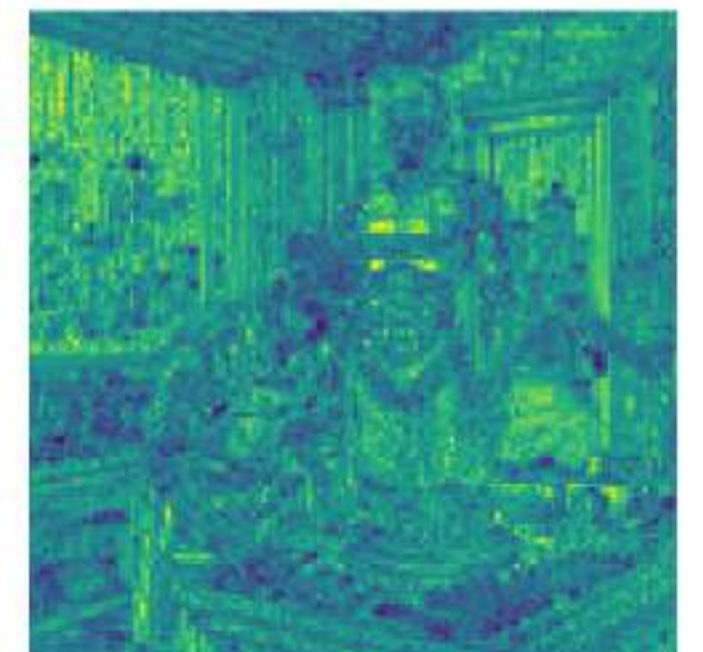
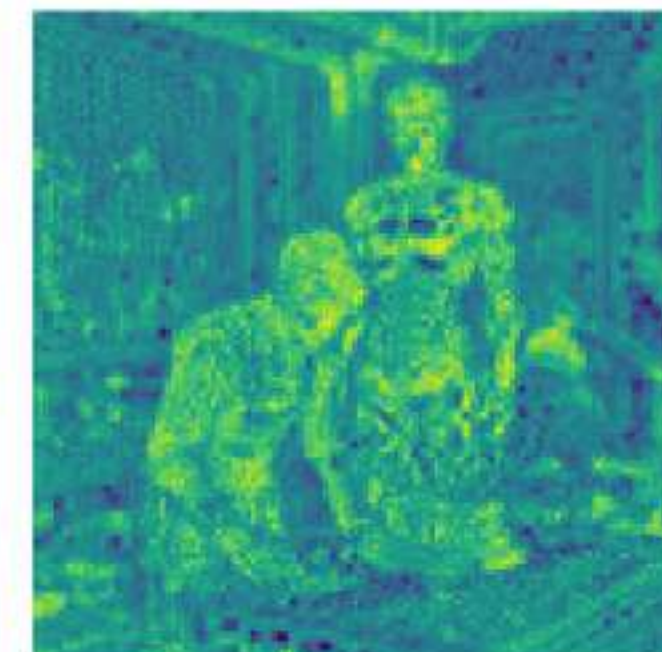
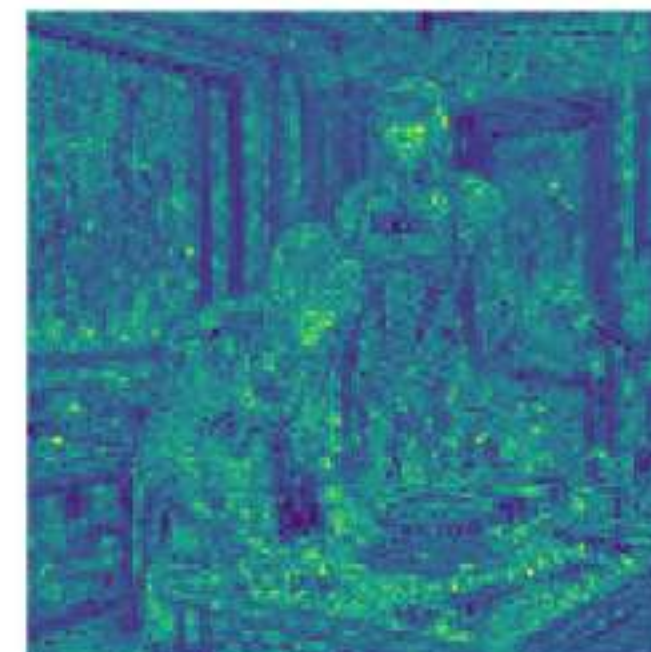
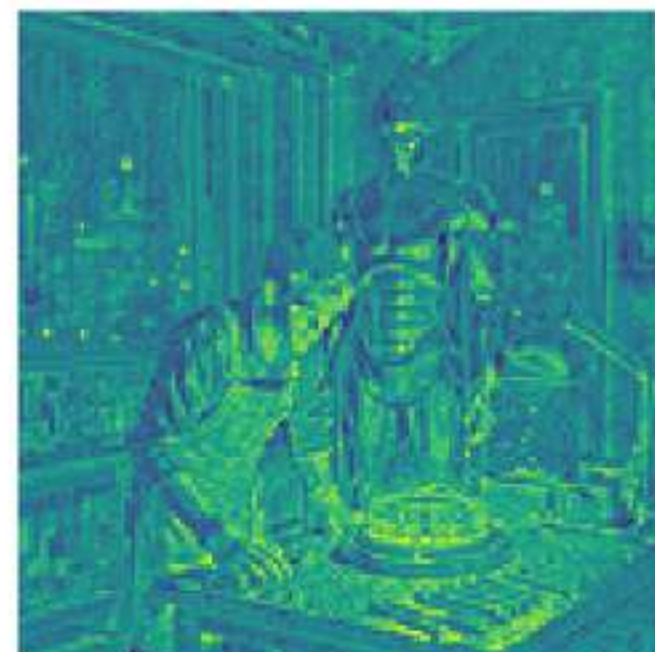
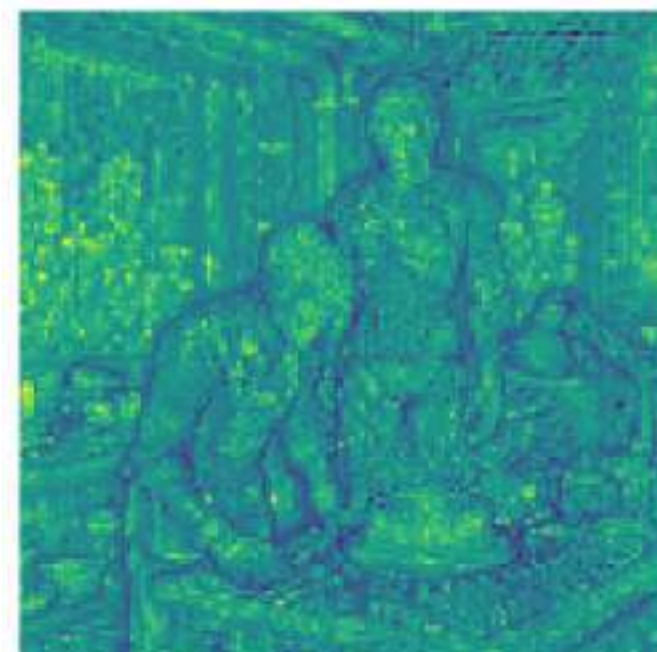
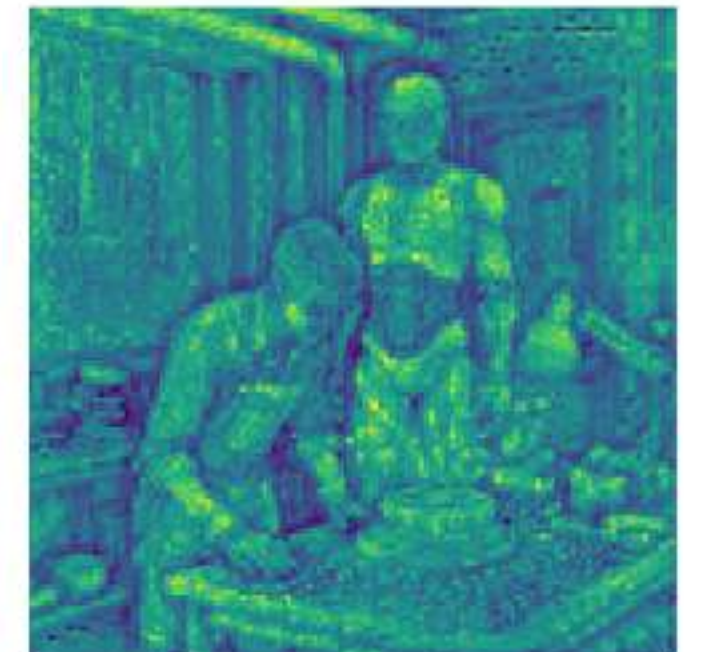
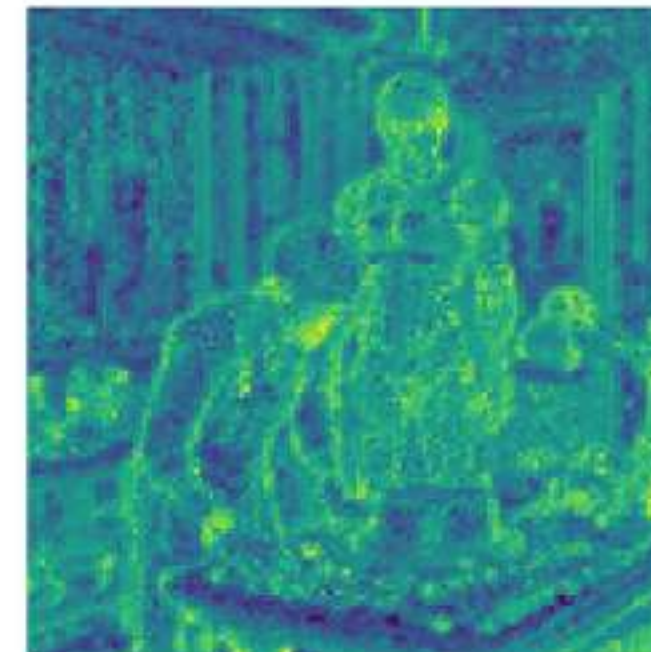
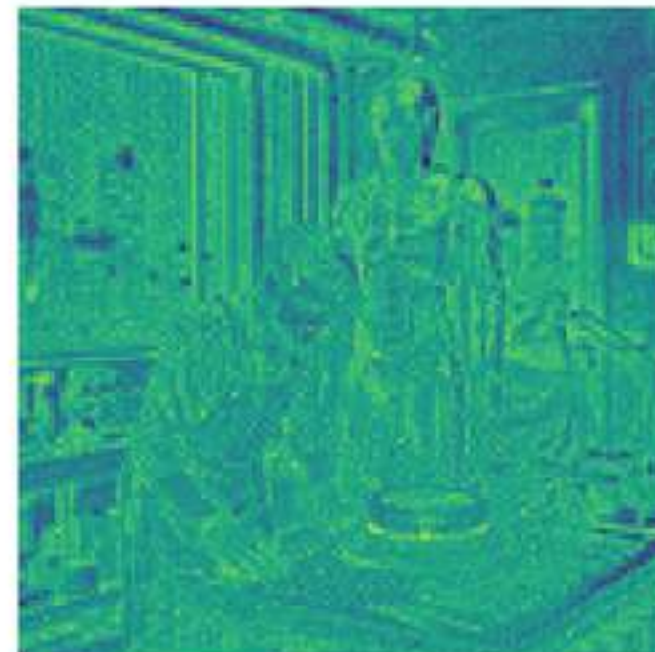
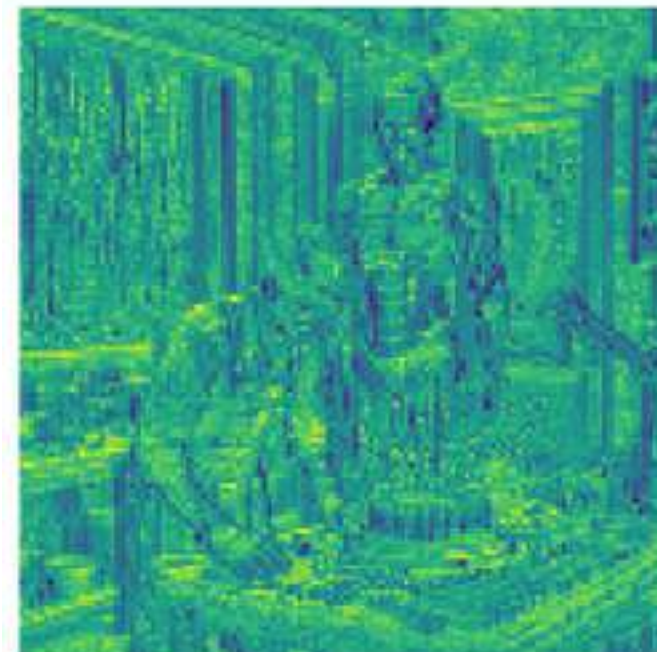
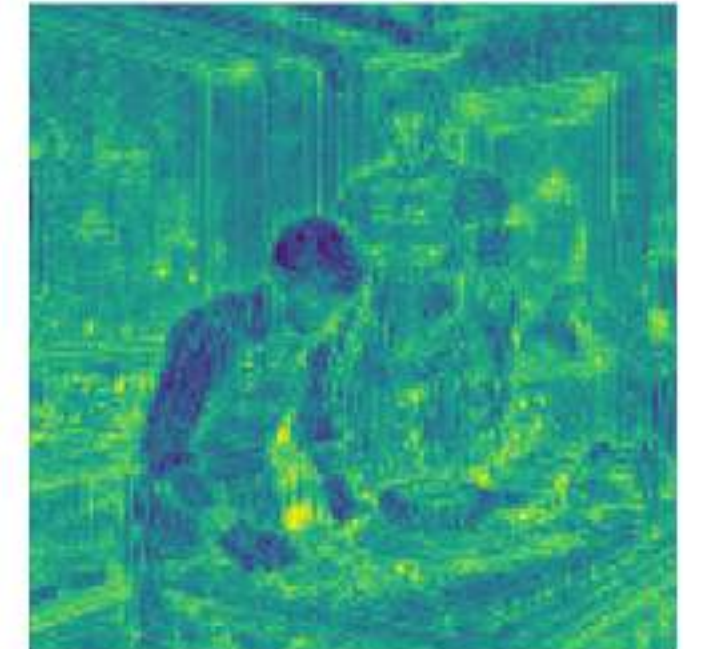
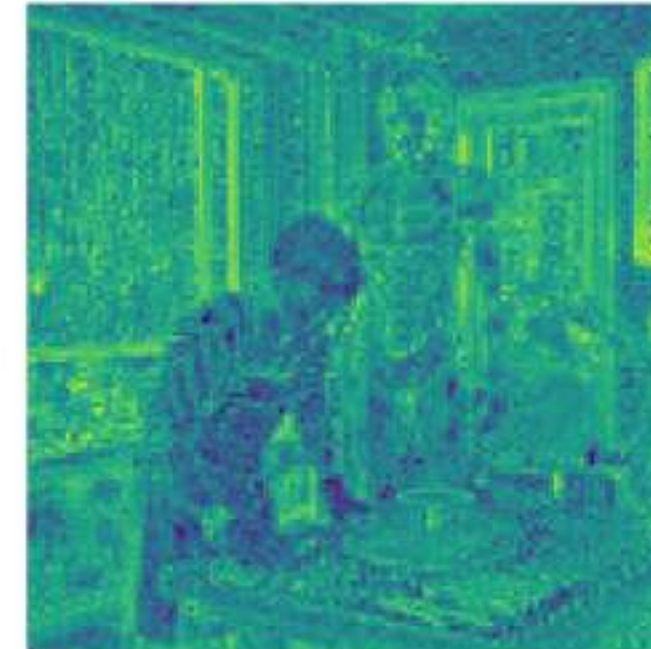
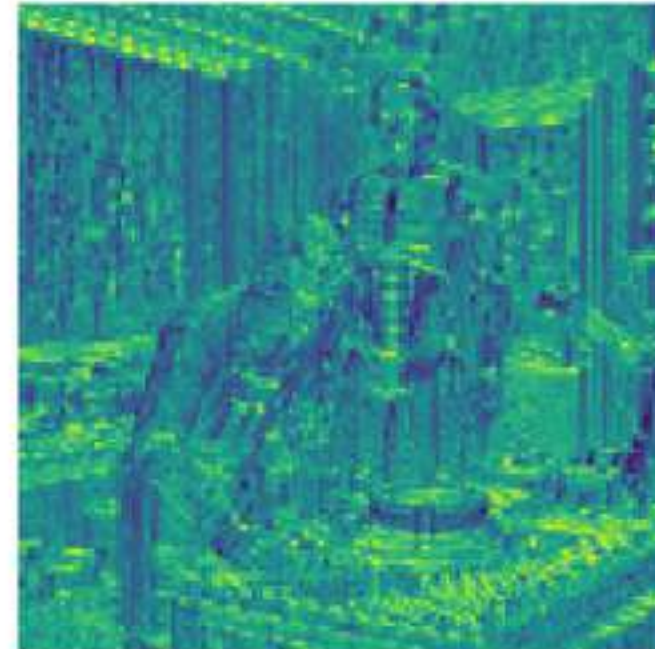
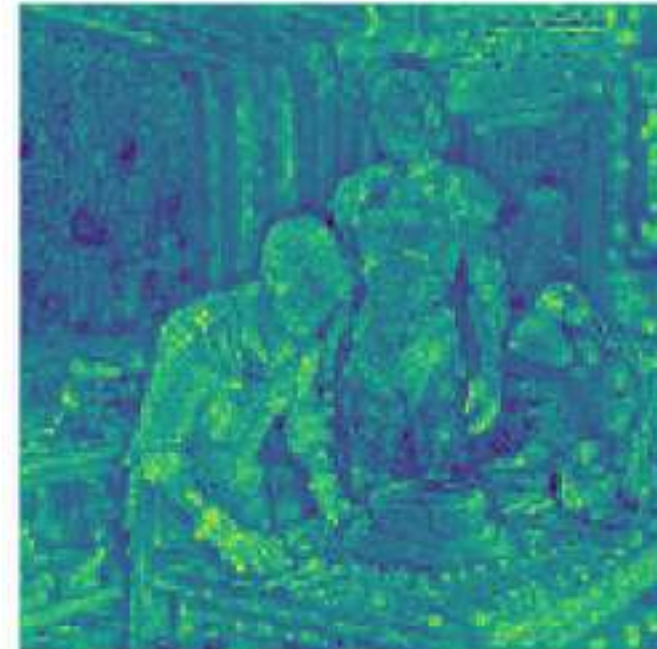
Reconstruction AE loss: $L_{AE} = L_{rec} + L_{lpips} + L_{GAN}$

FLUX.2 VAE latent space

Image



VAE latent channels



Latents are more like
advanced pixels

Quality-Learnability-Compression trade-off

What do we want from a VAE for diffusion?

Quality (Top priority)

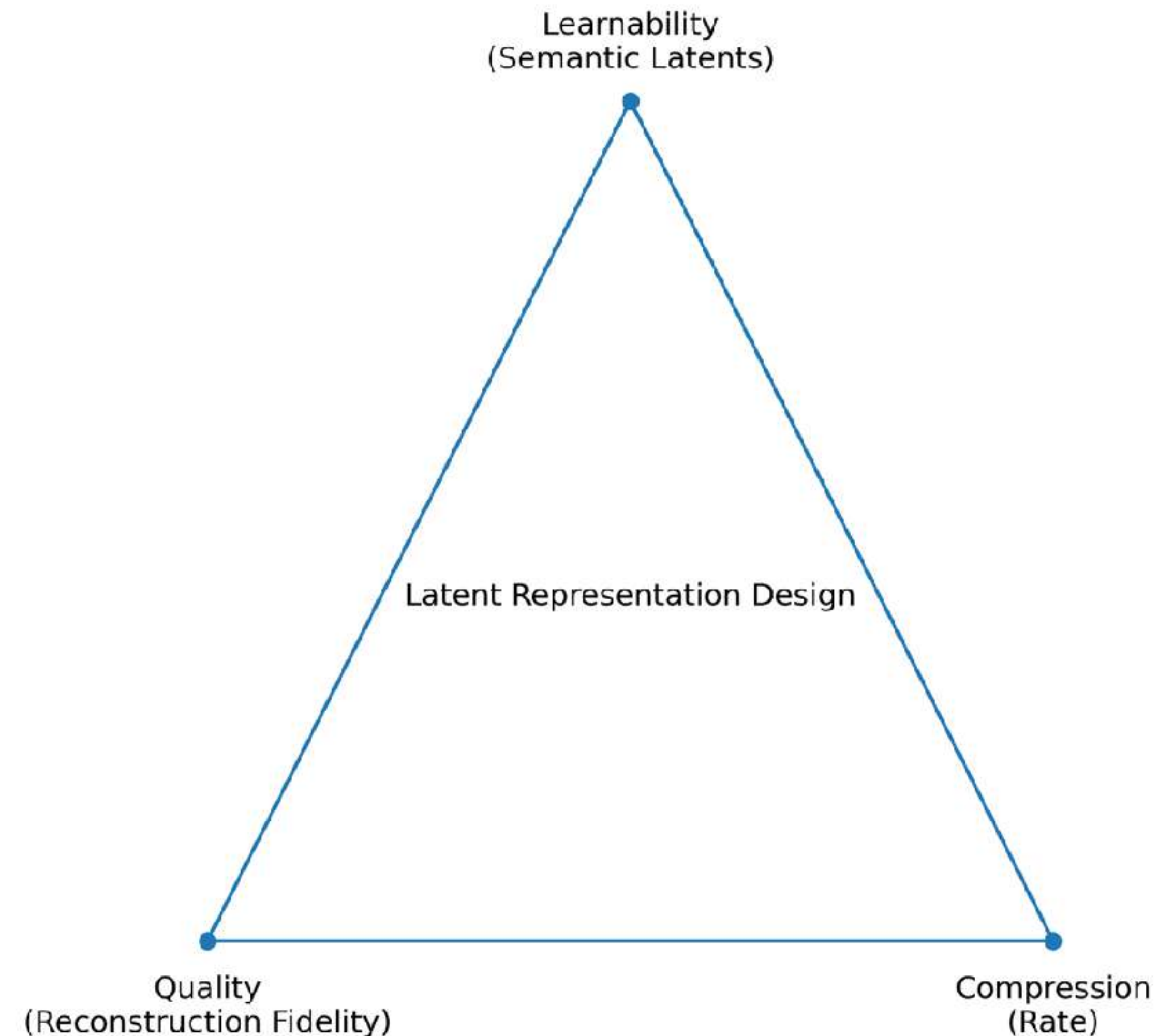
Nearly perfect image reconstruction

Learnability

Semantically structured latents make diffusion easier to learn

Compression

Lower-dimensional latent space for computational efficiency



How to enforce learnability?

The same image can be encoded in many different ways → find the more convenient one for diffusion

Apply latent space regularization

Examples

1. **VAE-like prior:** $D_{KL}(p_{\theta}(z) \parallel \mathcal{N}(0, I))$ — make latents closer to standard Gaussian
2. **Scale equivariance:** similar latent at different image resolutions
3. **Representation losses:** bias the encoder toward more **semantic structure**

Takeaways

Pixel-space models are promising and actively developed at the moment

$\mathbf{x}_0$ -prediction is critical in pixel space. In latent space, network prediction type is less important

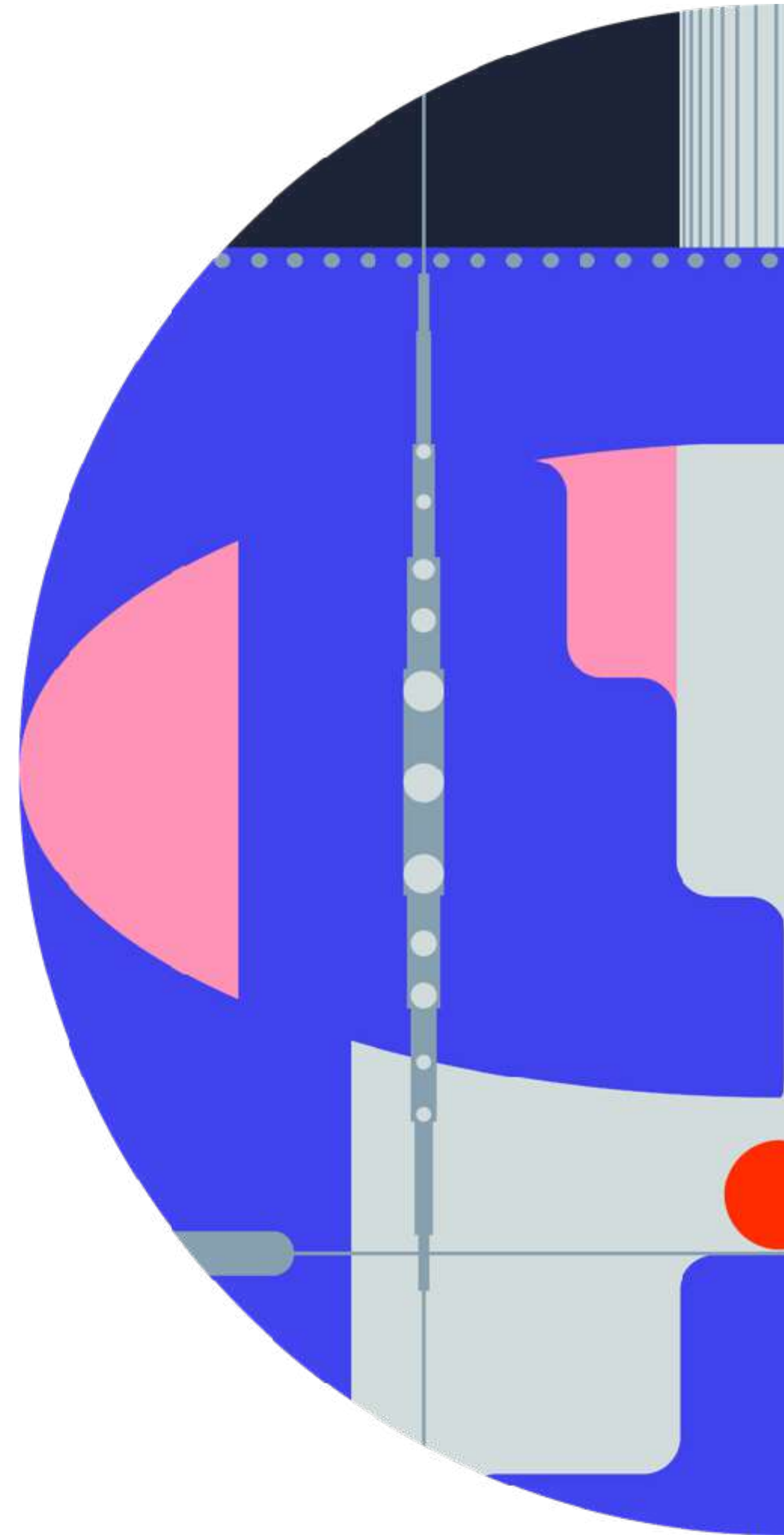
Latent diffusion models are a default choice in production-grade models

AE takes care of **imperceptible noise** and **high-frequency details** → low-dim latent space → **higher learnability**

AE latent space can be regularized to pack images in a more convenient way for diffusion

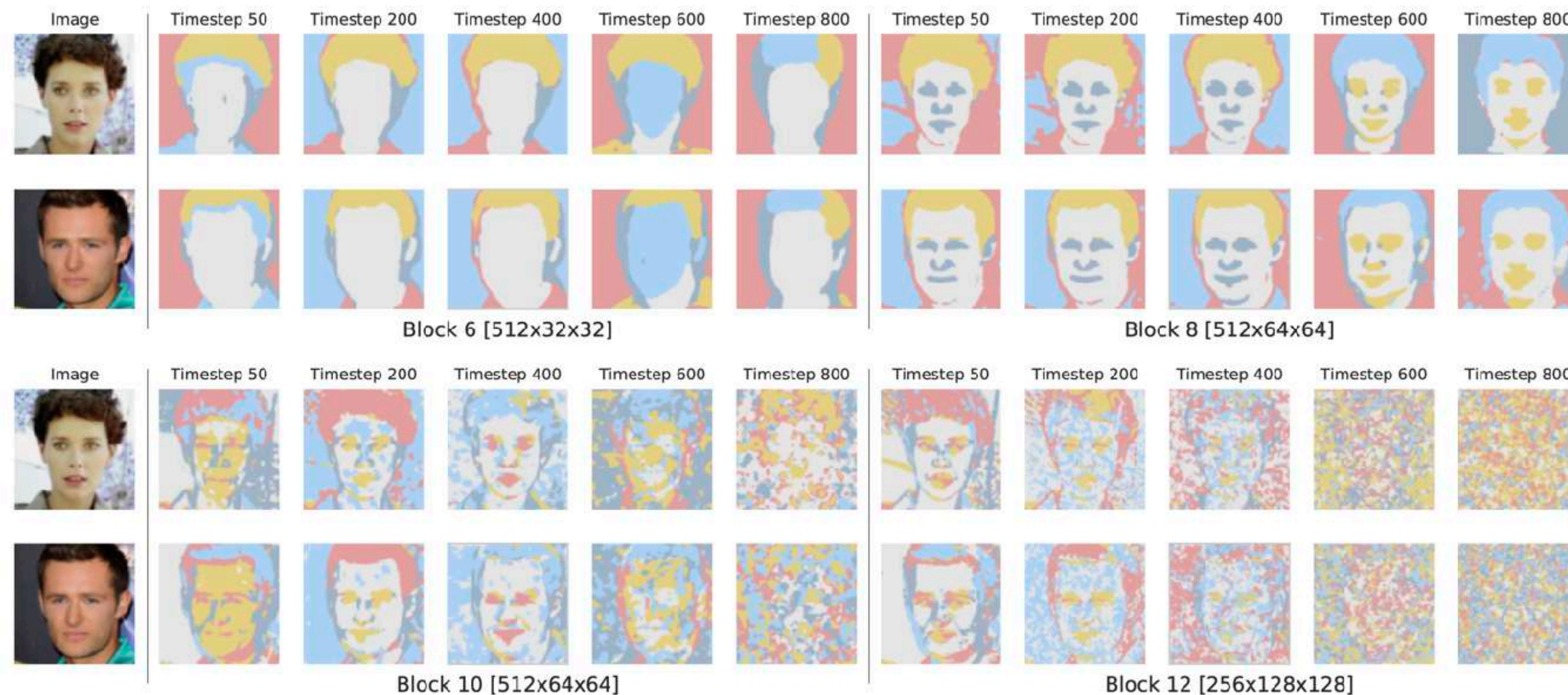
Designing AE with **nearly perfect reconstruction** and **high learnability** is a high-priority problem today

Diffusion models as representation learners

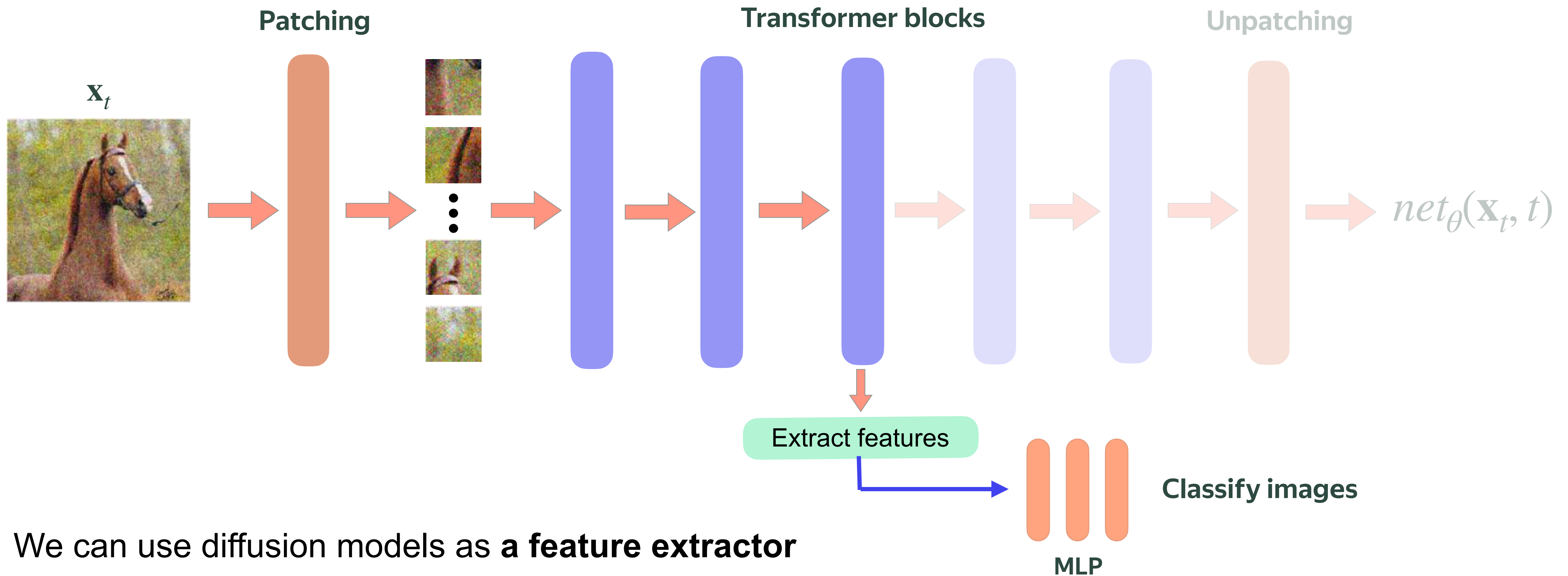


Diffusion model as representation learners

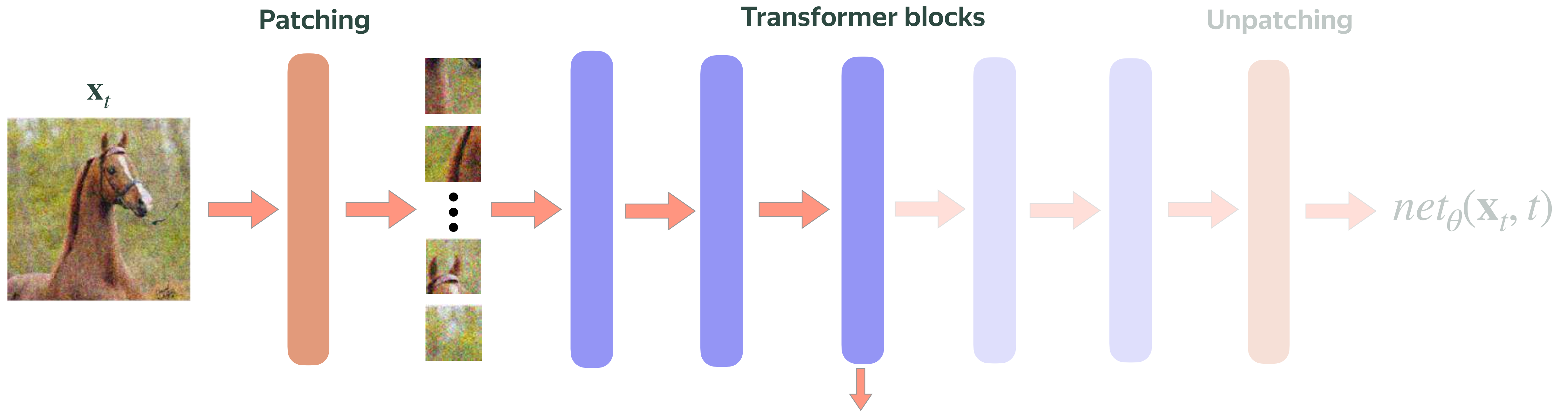
Diffusion models learn high quality image internal representations



Diffusion model as representation learners



Diffusion model as representation learners

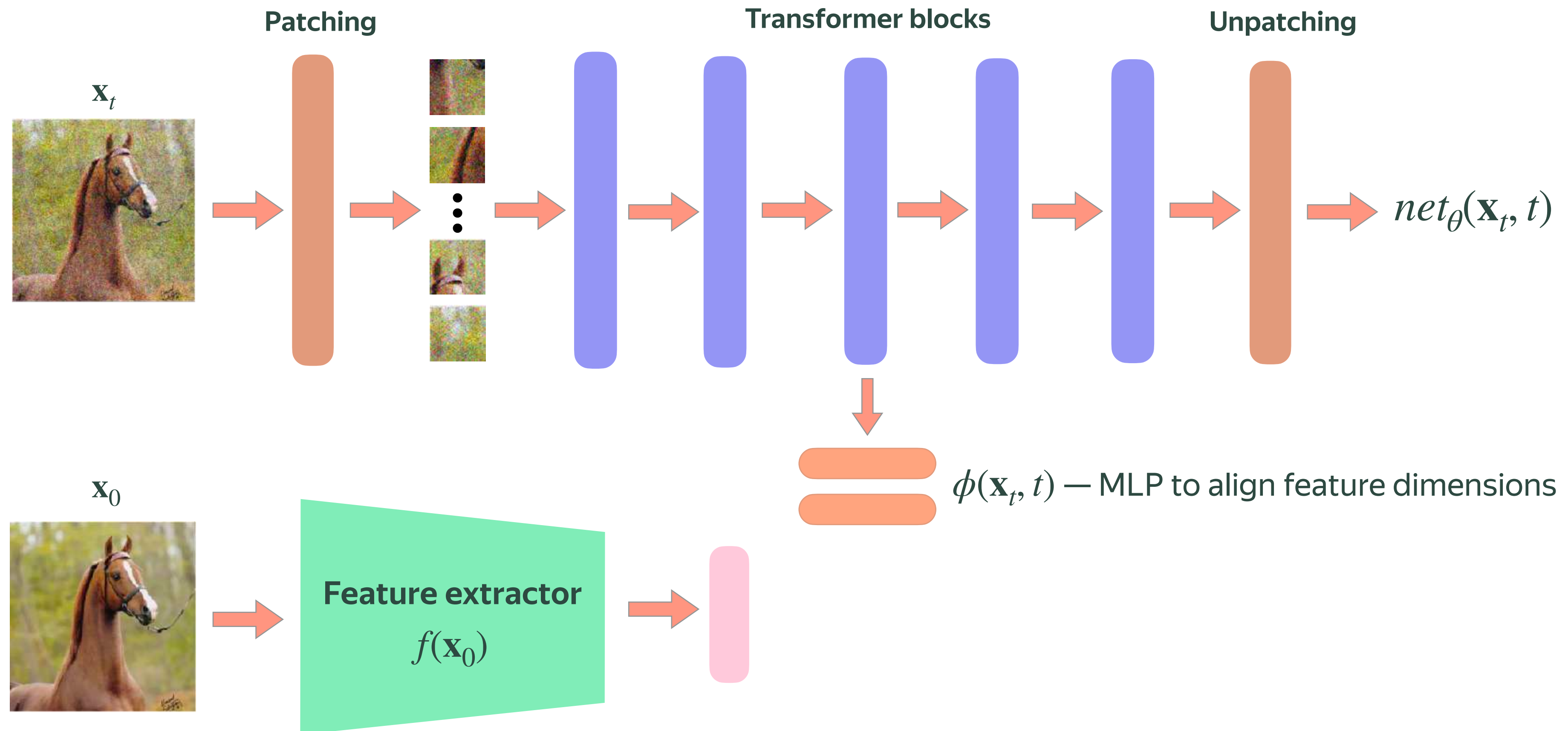


Is it difficult to learn good image representations? **Yes!**

Diffusion models spend much training time on representation learning

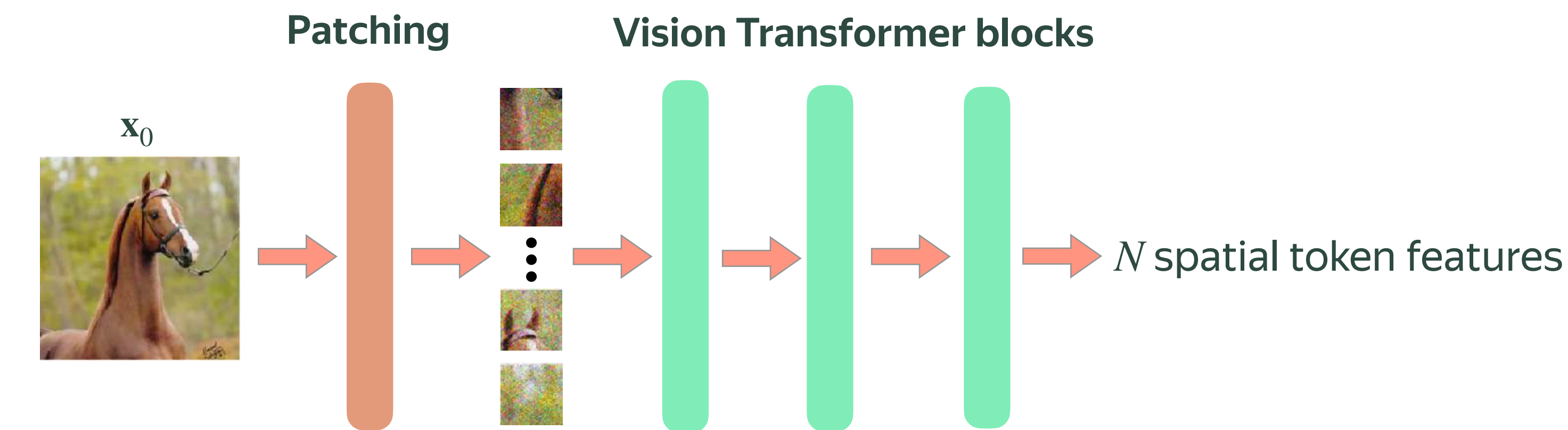
Representation Alignment | REPA

Idea: align diffusion features with a pretrained feature extractor



Pretrained feature extractor

DINOvX — state-of-the-art image feature extractors today



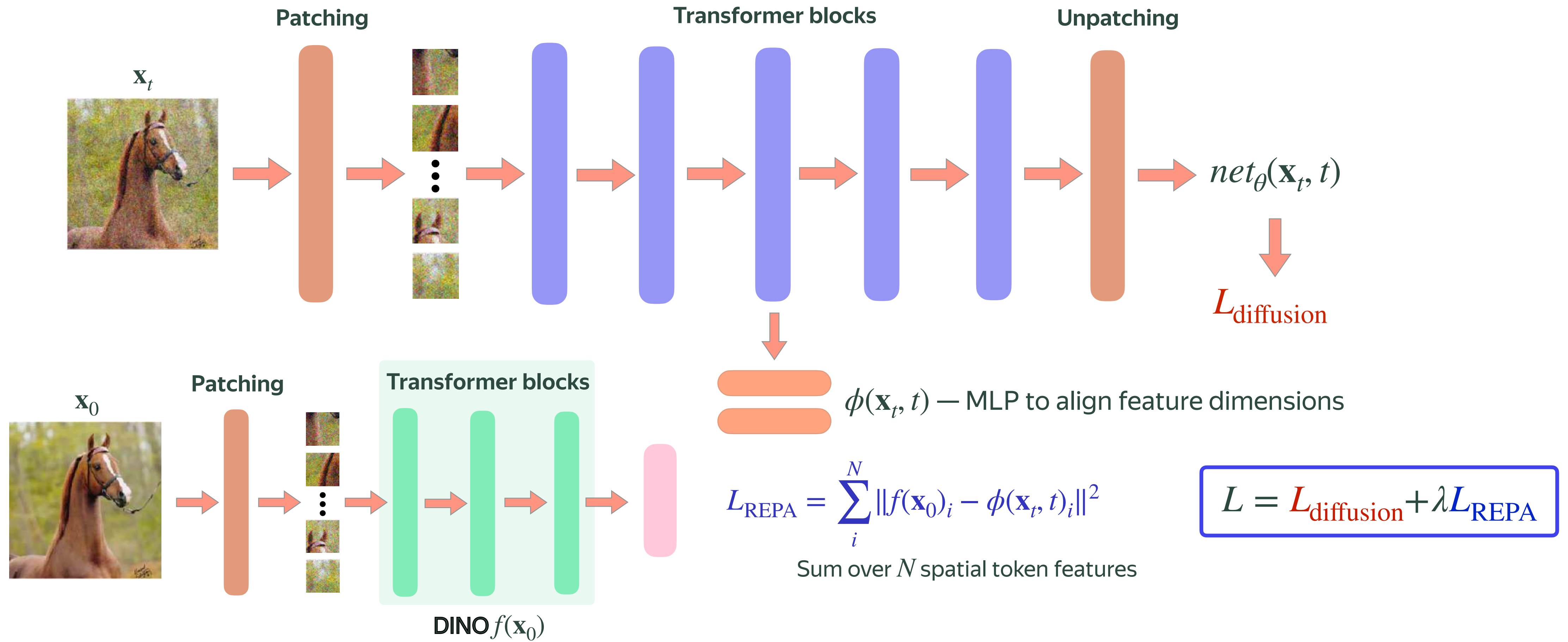
Image



DINOv3 features

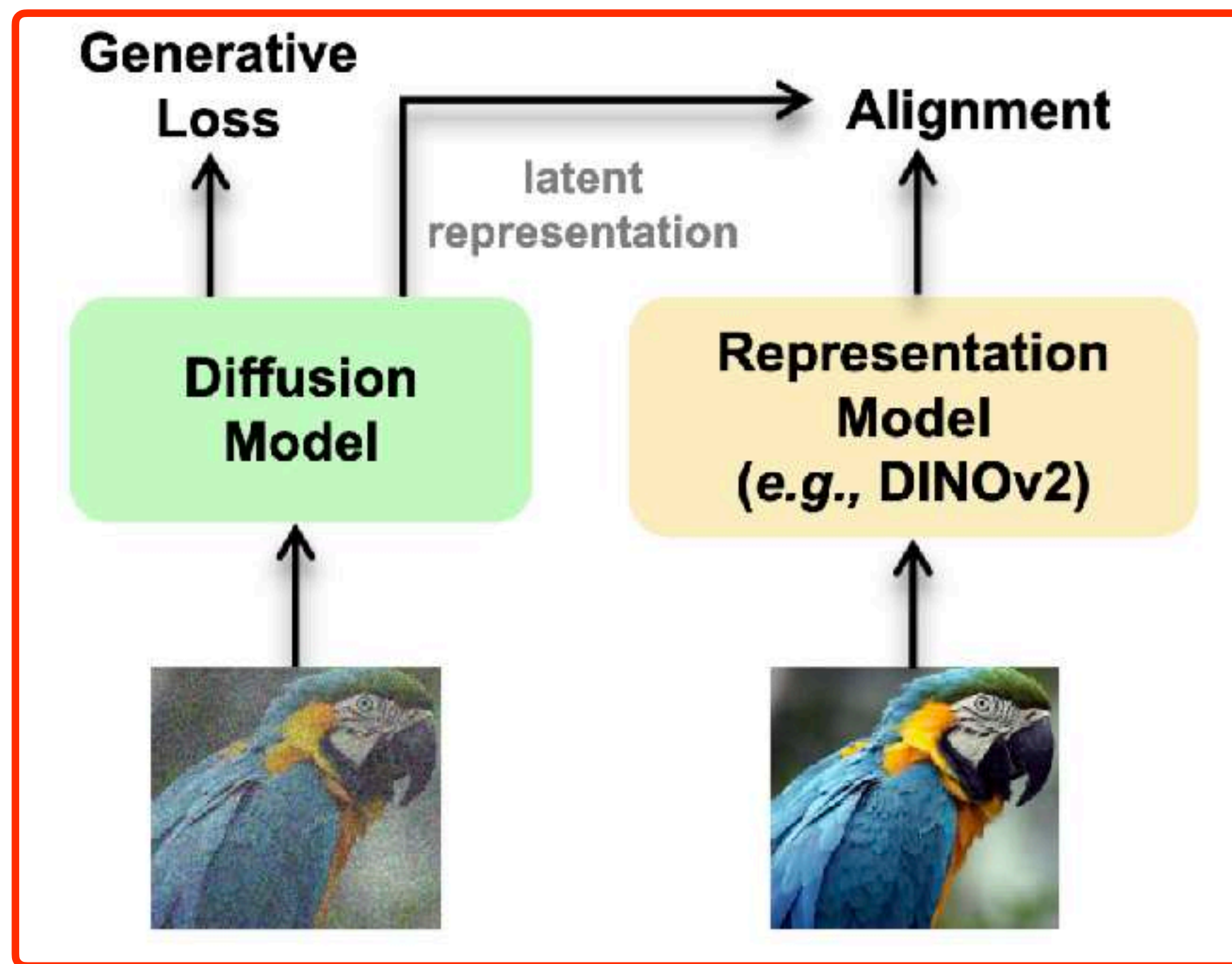


REPA with DINO



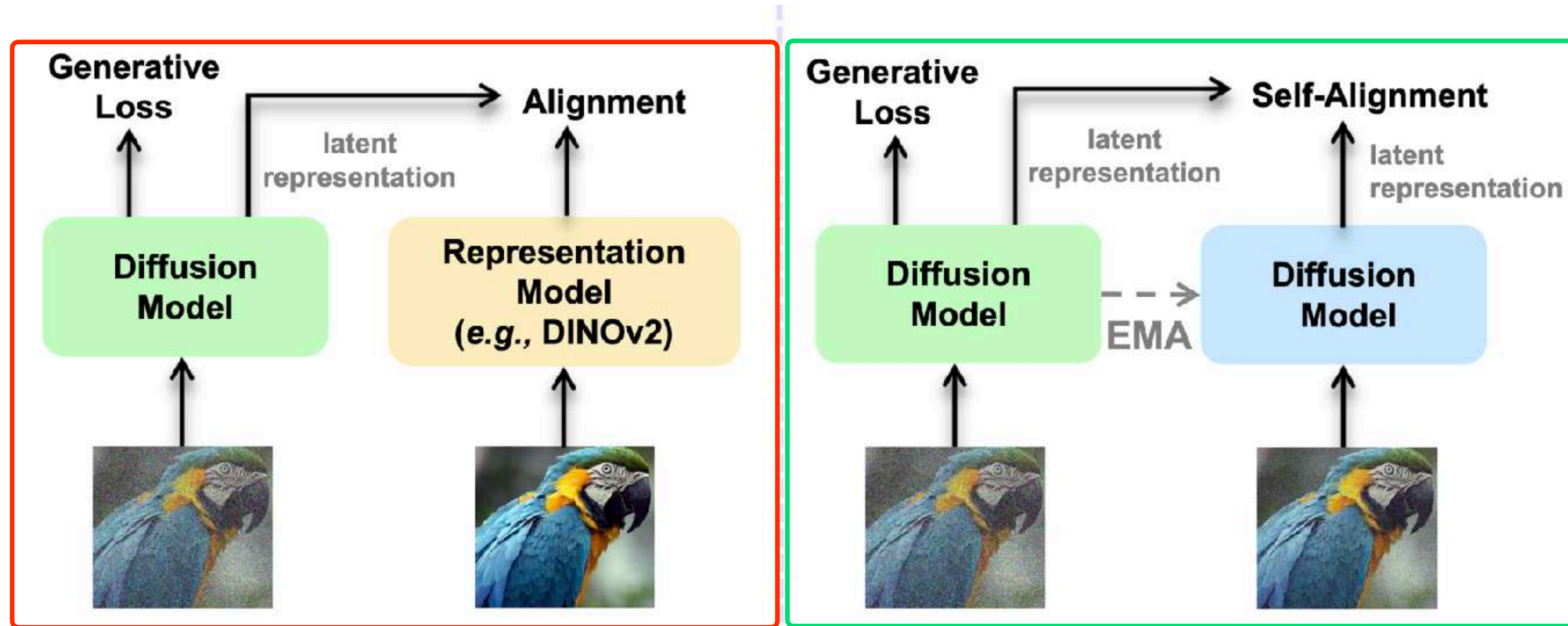
REPA problem

REPA with external encoders does not scale well for large text-to-image models



Self-alignment

Self-alignment: align diffusion features with those from its EMA version

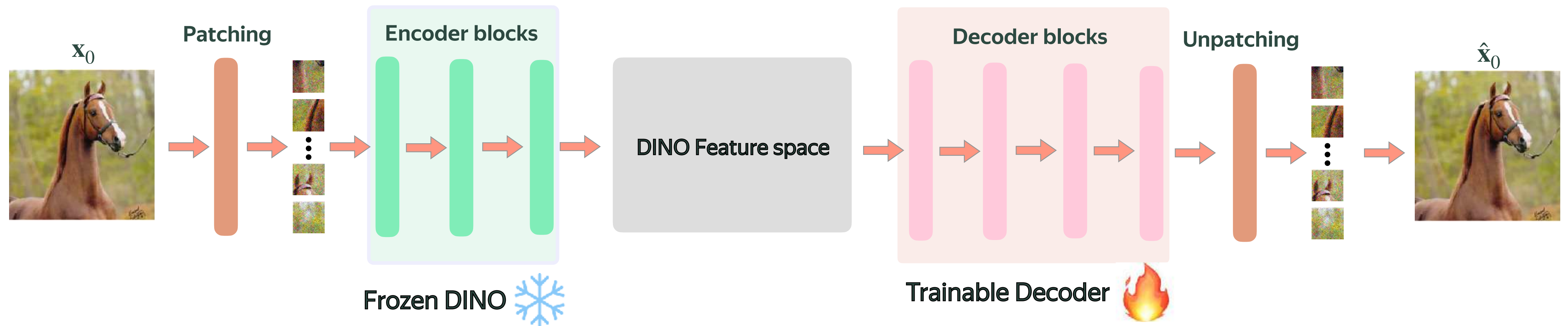


RAE | Representation autoencoder

If DINO helps diffusion training, maybe **train it directly in DINO feature space?**

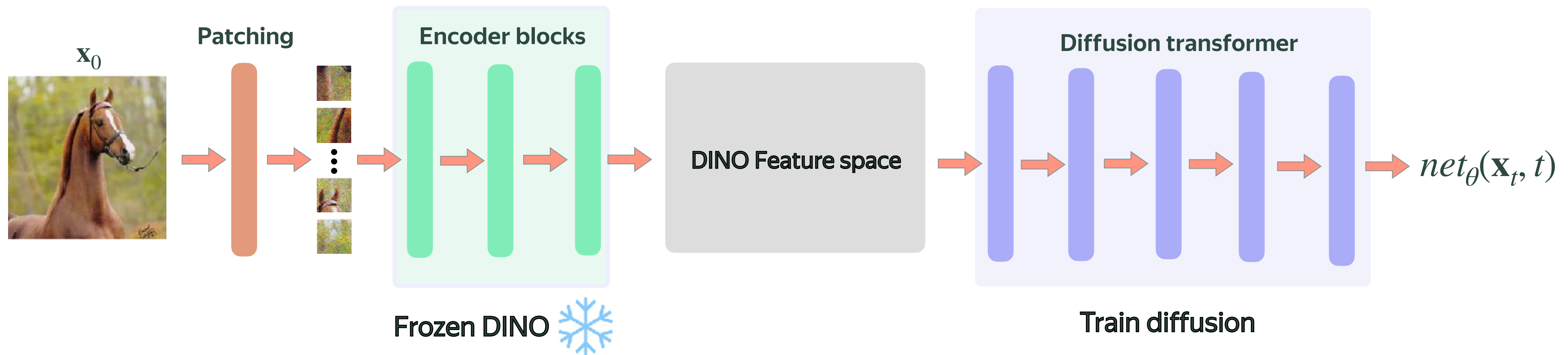
RAE | Training autoencoder

1) Train AE, where DINO is a frozen encoder



RAE | Training diffusion

2) Train diffusion in DINO feature space



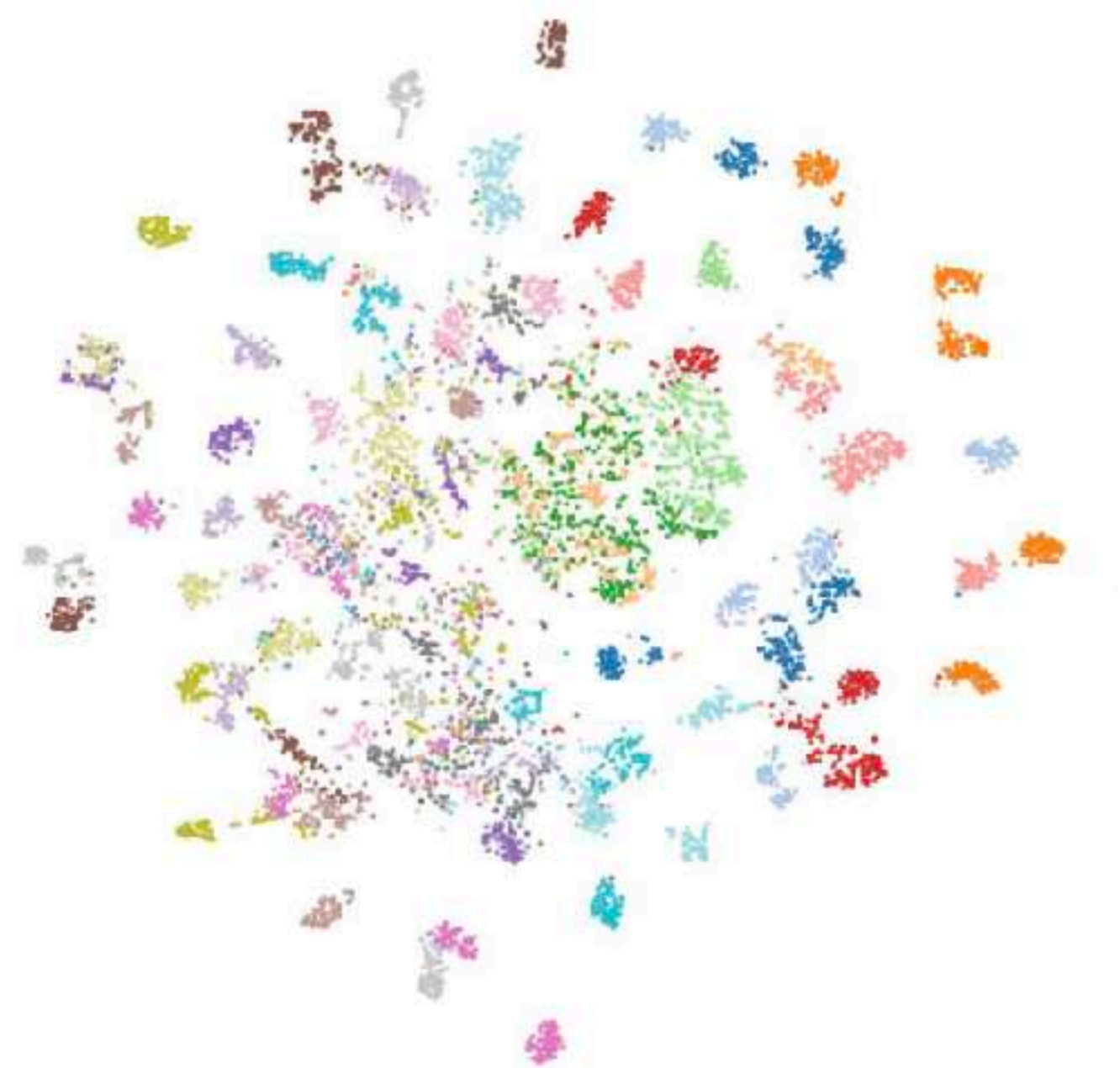
Why RAE?

DINO provides highly structured latent space

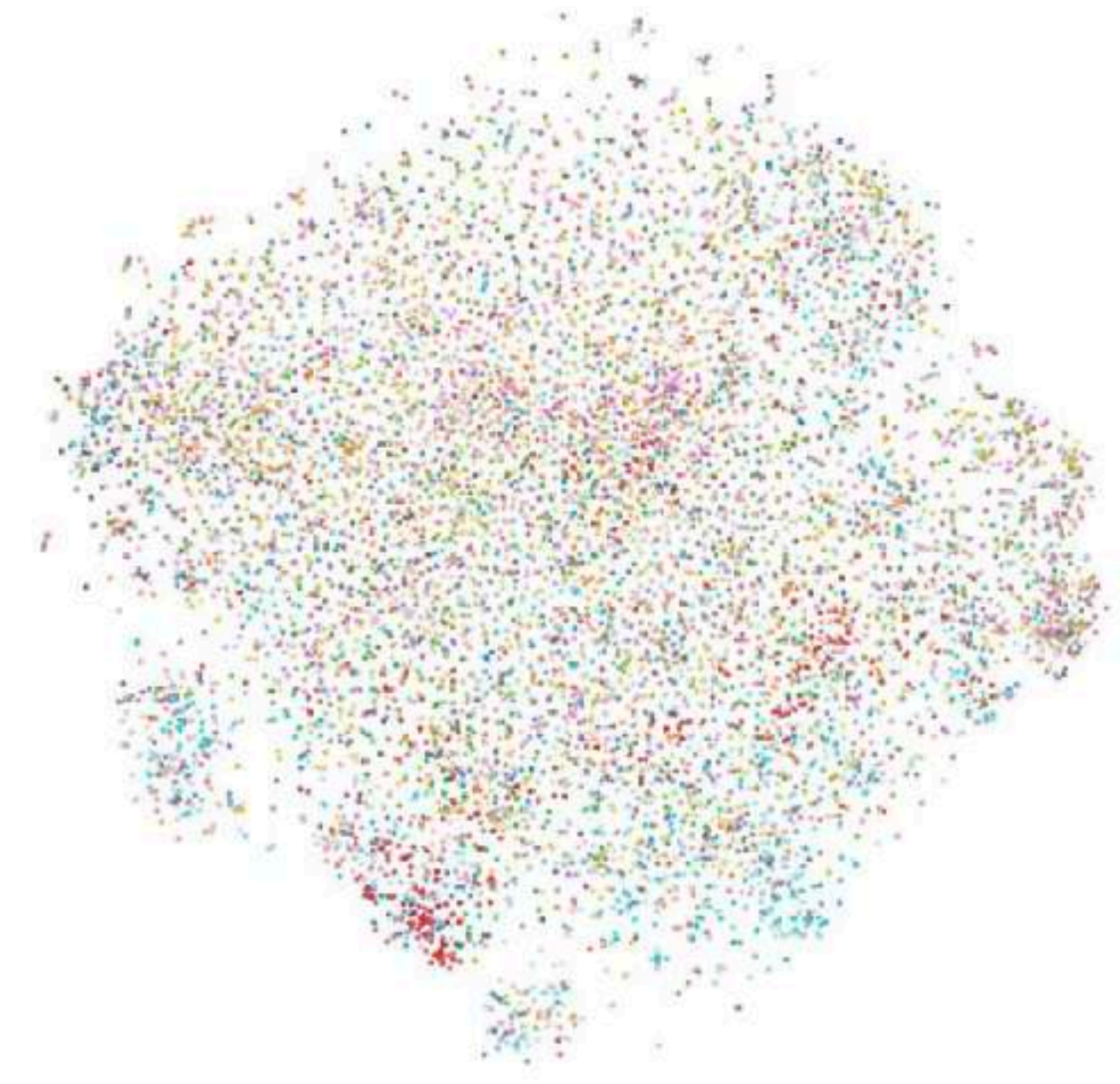
Very high learnability

Poor reconstruction quality
(Work in progress)

Latent space structures



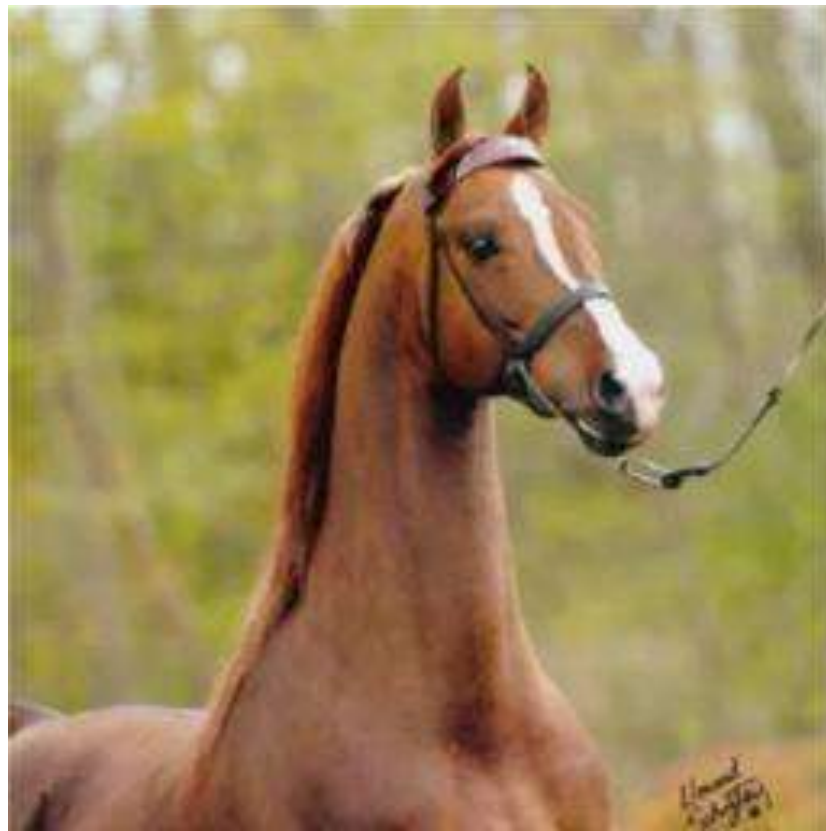
DINOv3



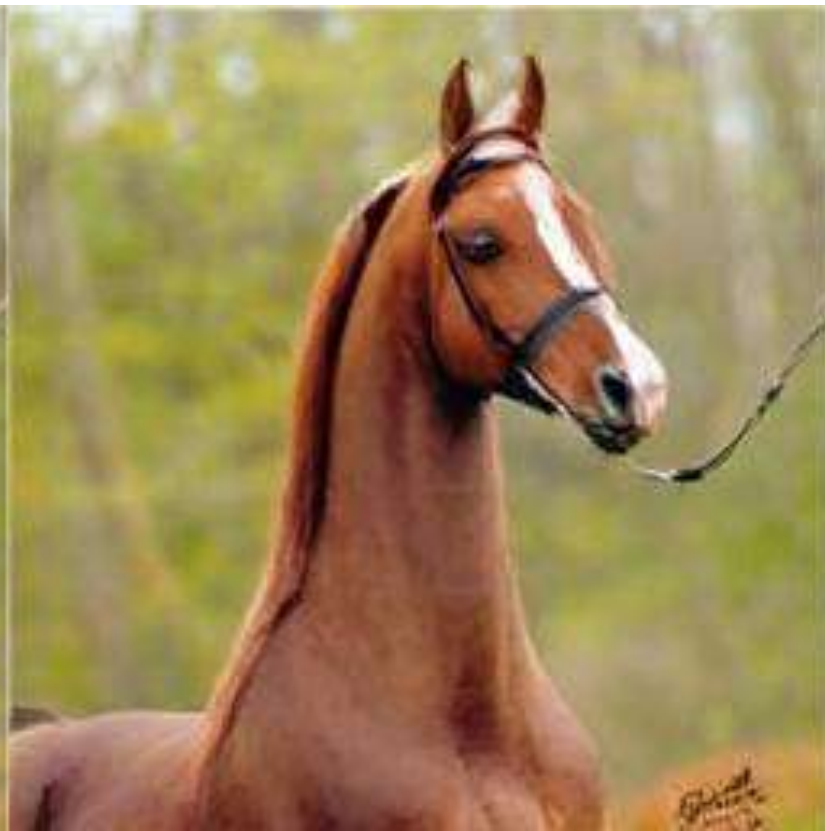
SD-VAE

Reconstruction examples

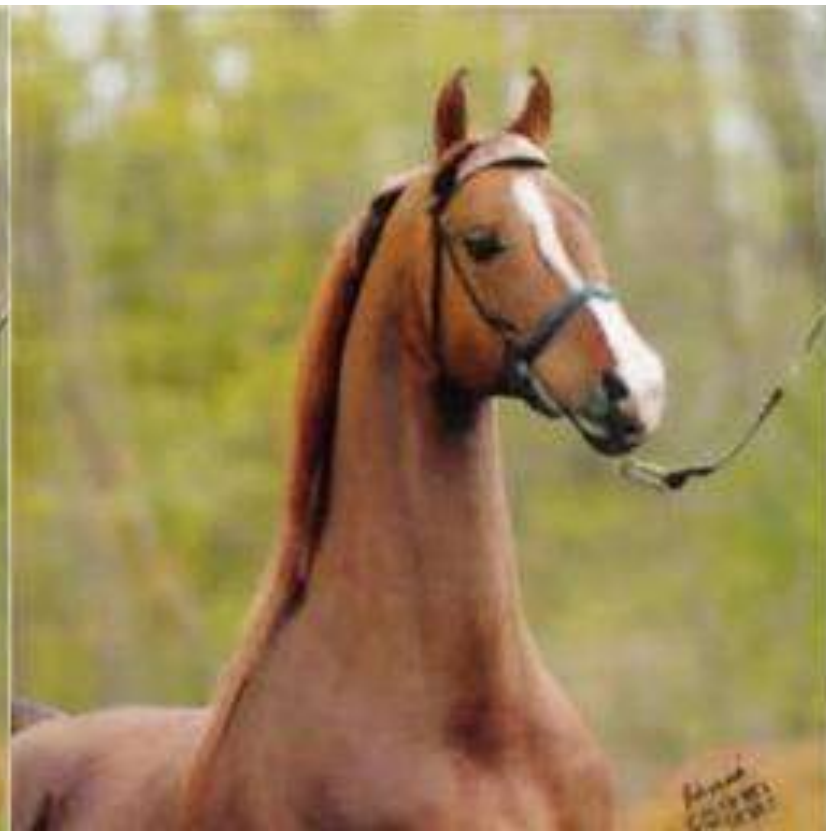
Image



RAE



SD-VAE



Image



RAE



SD-VAE



RAE has problems with low-level (colors) and fine-grained details

Takeaways

Diffusion models learn **strong semantic representations** in addition to generation

Diffusion models spend much training time to learning internal feature representations

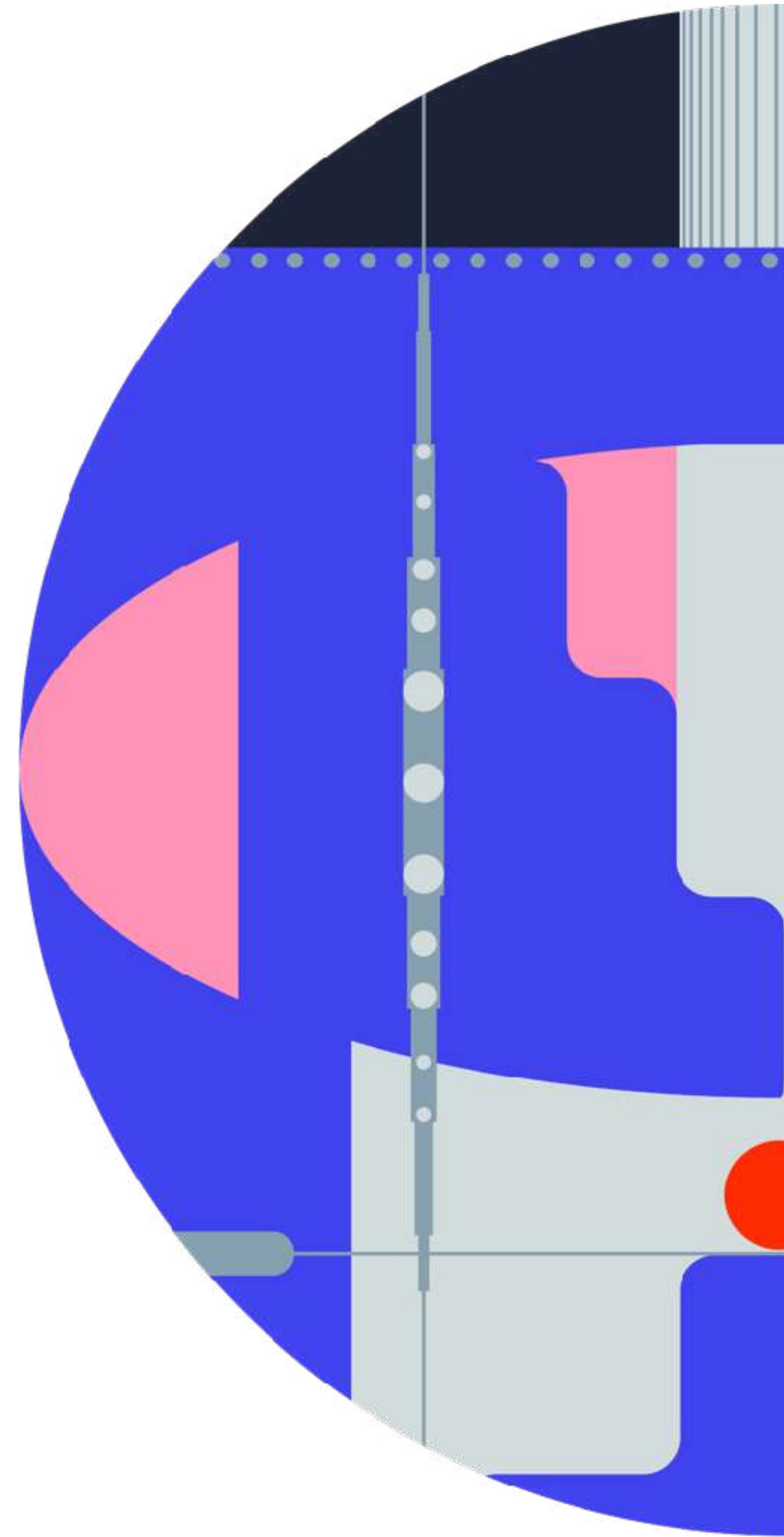
REPA improves and accelerates diffusion feature learning

In s.o.t.a. T2I, REPA + external encoders works poorly. Self-aligned diffusion features are promising

RAE enables training diffusion directly in semantic latent space

RAE offers **great learnability** but **poor reconstruction**; the direction is actively developing

How to sample with diffusion models?



Sampling schedule

$$s(\alpha, \cdot) : t \mapsto \frac{\alpha t}{1 + (\alpha - 1)t}$$

Similar intuition as at training

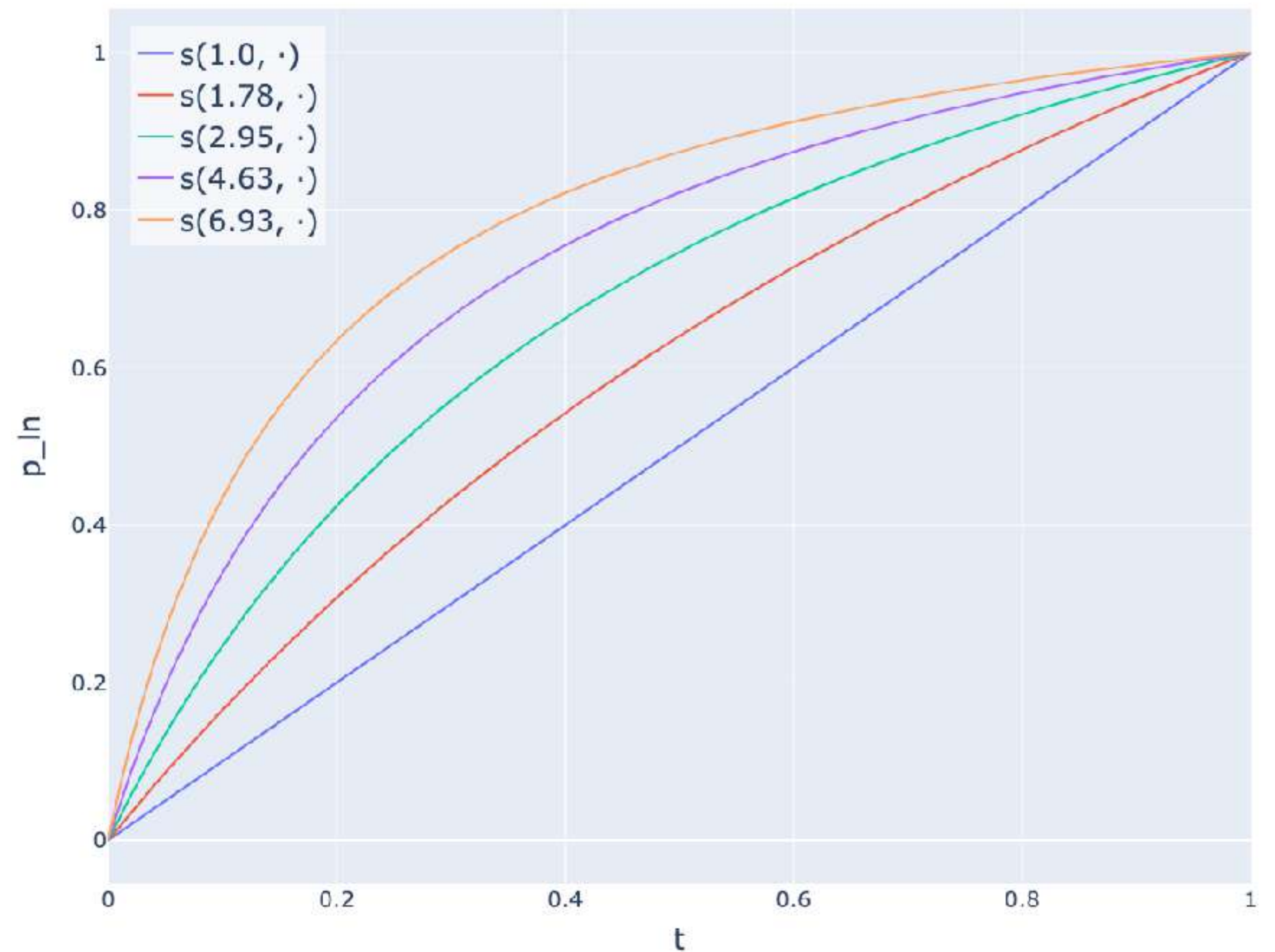


Higher resolution $\rightarrow$ larger shift α

$\alpha = 1$ — uniform sampling

Okay in low-dimensional spaces

Timeshift function



Why this specific $s(\alpha, \cdot)$?

Why exactly this $s(\alpha, \cdot) : t \mapsto \frac{\alpha t}{1 + (\alpha - 1)t}$?

Why this specific $s(\alpha, \cdot)$?

Recall our dummy variable $Y(t) = (1 - t)c + t\epsilon$,

$$\text{Var} [Y(t)] = \frac{t^2}{(1 - t)^2} \frac{1}{N} \rightarrow \sigma(t, N) = \frac{t}{1 - t} \sqrt{\frac{1}{N}}$$

Let's fix N and increase image size from N to M

What is t_{new} to make $\sigma(t, N) = \sigma(t_{new}, M)$?

Why this specific $s(\alpha, \cdot)$?

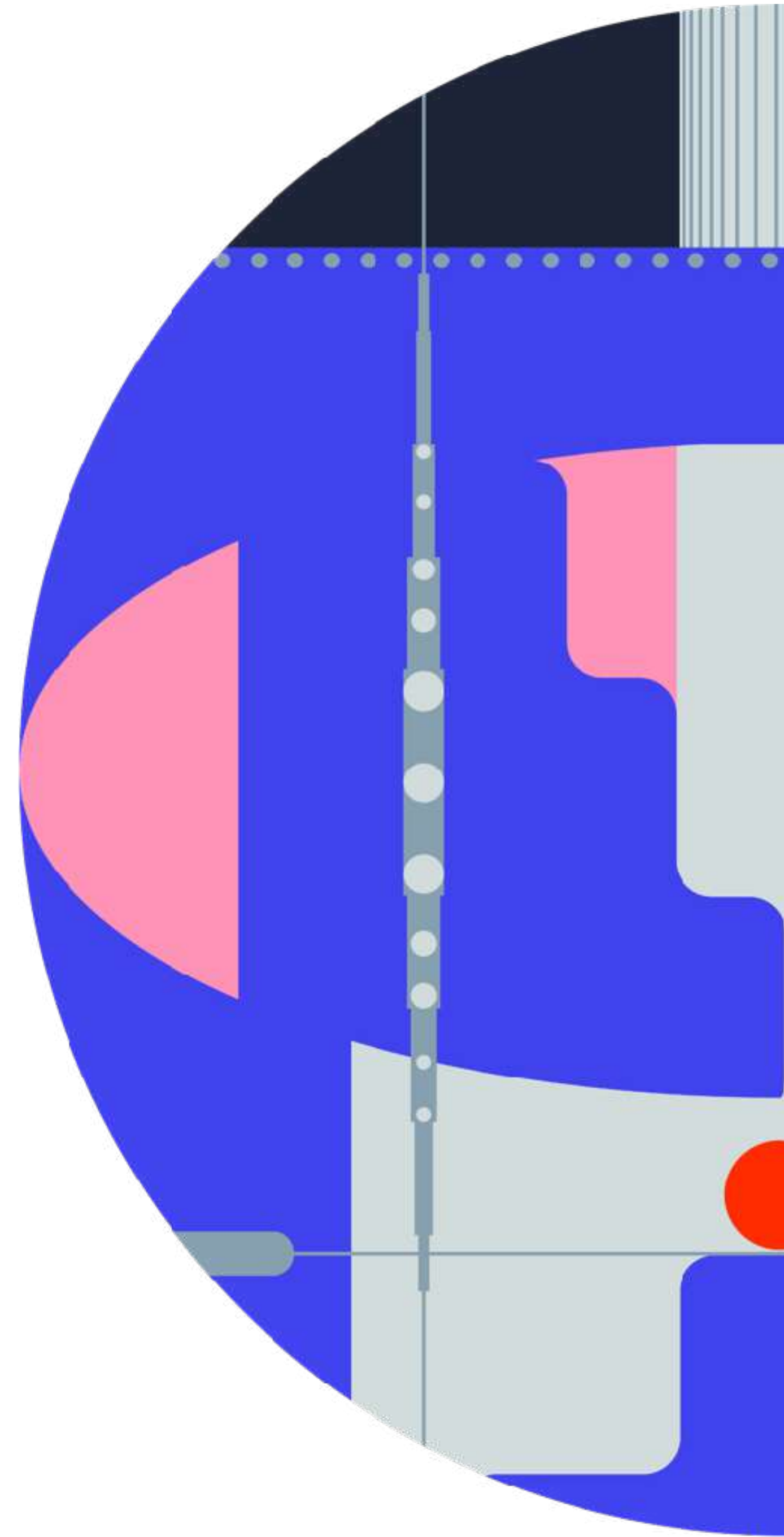
$$\sigma(t_{new}, M) = \sigma(t, N) \rightarrow \frac{t_{new}}{1 - t_{new}} \sqrt{\frac{1}{M}} = \frac{t}{1 - t} \sqrt{\frac{1}{N}}$$

$$\frac{1}{1 - t_{new}} - 1 = \frac{t}{1 - t} \sqrt{\frac{M}{N}} \quad \rightarrow \quad 1 - t_{new} = \frac{1}{1 + \frac{t}{1 - t} \sqrt{\frac{M}{N}}} \quad \rightarrow \quad t_{new} = \frac{\frac{t}{1 - t} \sqrt{\frac{M}{N}}}{1 + \frac{t}{1 - t} \sqrt{\frac{M}{N}}}$$

$$t_{new} = \frac{t \sqrt{\frac{M}{N}}}{1 + t(\sqrt{\frac{M}{N}} - 1)} = \frac{\alpha t}{1 + (\alpha - 1)t}, \quad \text{where } \alpha = \sqrt{\frac{M}{N}}$$

α controls how the schedule shifts
when scaling resolution $N \rightarrow M$
from a uniform optimal schedule

Advanced guidance techniques



Classifier-free Guidance

CFG is critical for all practical generation scenarios

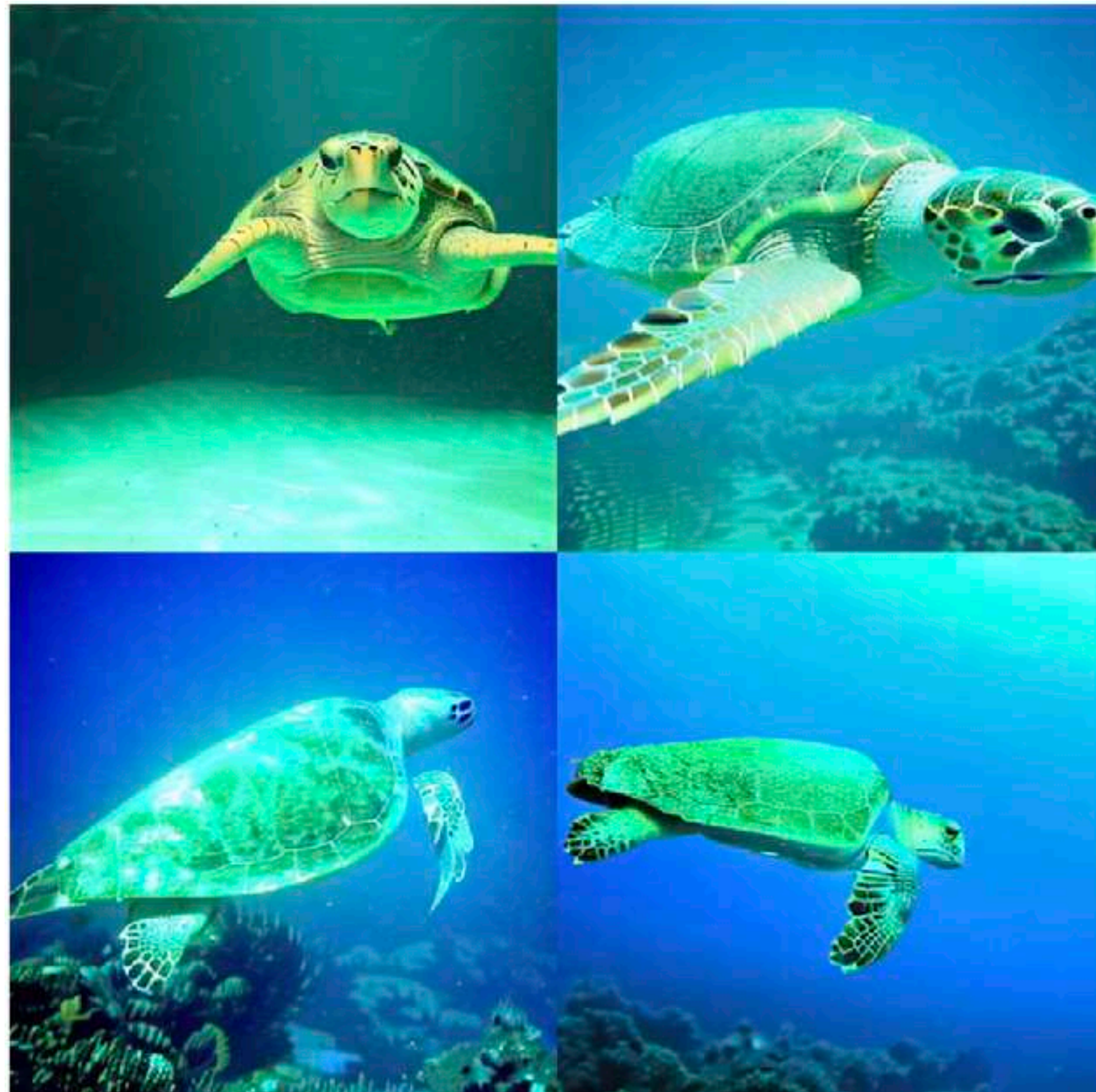
$$\mathbf{v}_{CFG}(\mathbf{x} \mid \mathbf{c}) = \mathbf{v}_{\theta}(\mathbf{x}, \emptyset) + w \cdot (\mathbf{v}_{\theta}(\mathbf{x}, \mathbf{c}) - \mathbf{v}_{\theta}(\mathbf{x}, \emptyset)), \quad w \geq 1$$

Improves condition alignment and quality

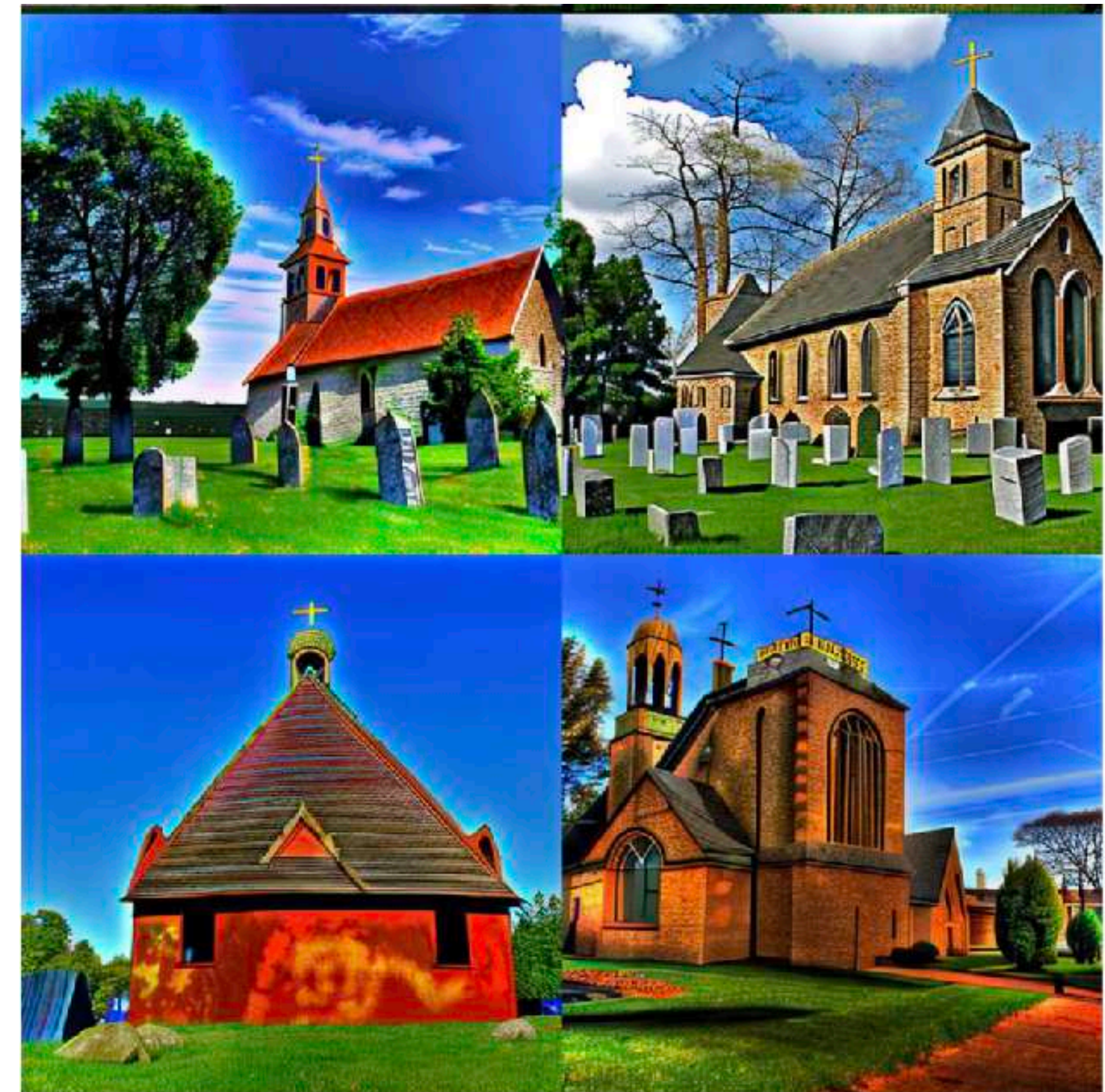
Reduces diversity and oversaturates images for large guidance scales

High CFG problems

Low diversity & Oversaturation

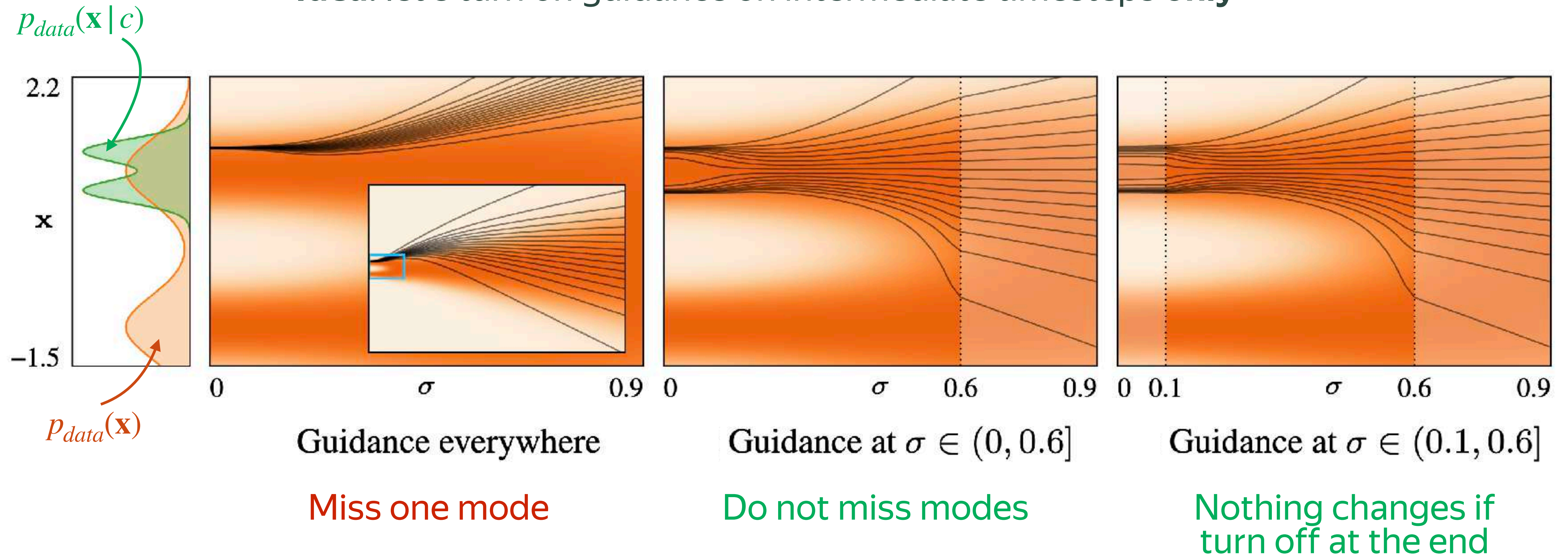


Oversaturation



Interval Guidance

Idea: let's turn on guidance on intermediate timesteps **only**



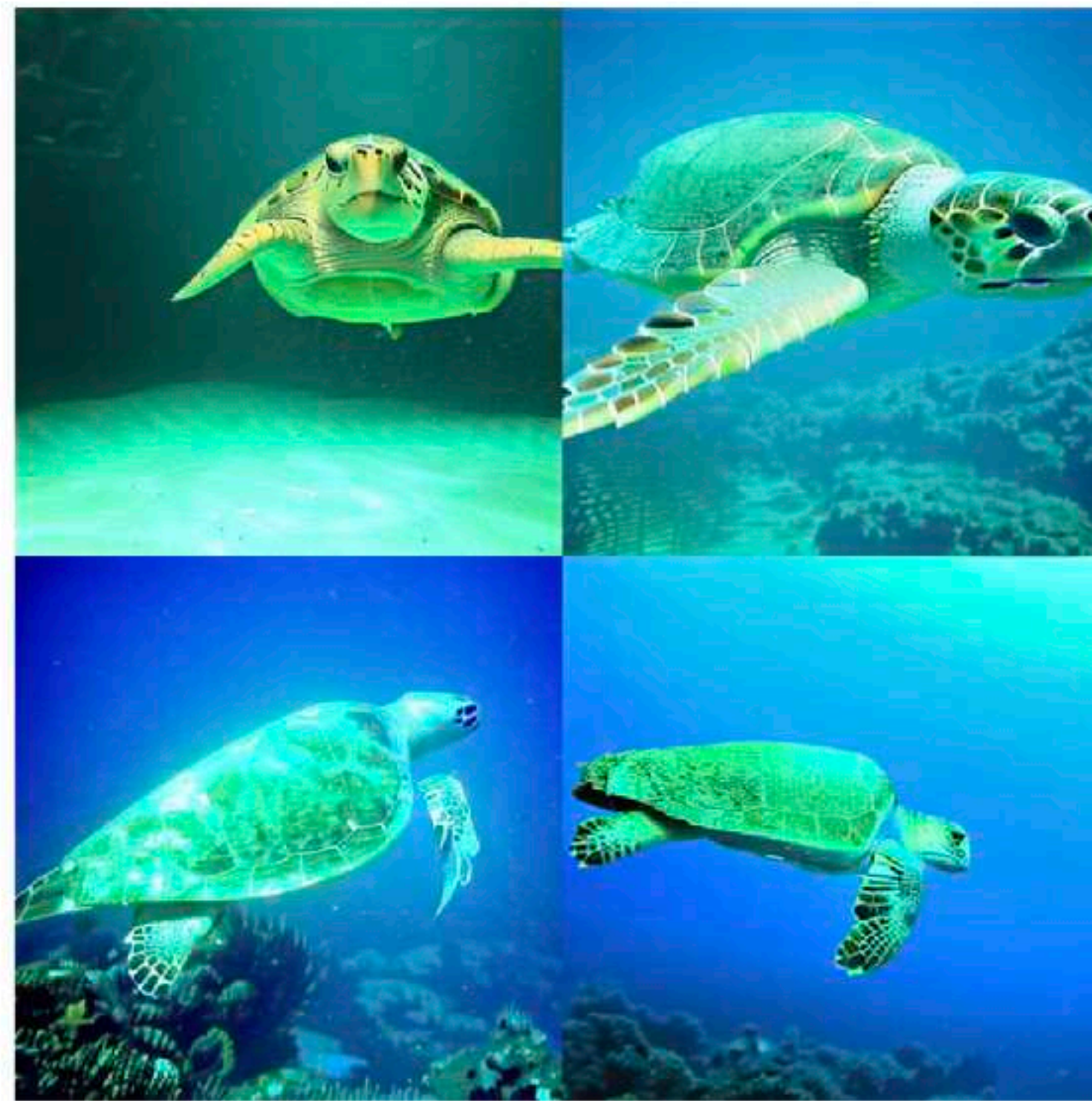
Interval Guidance



Interval Guidance



No CFG

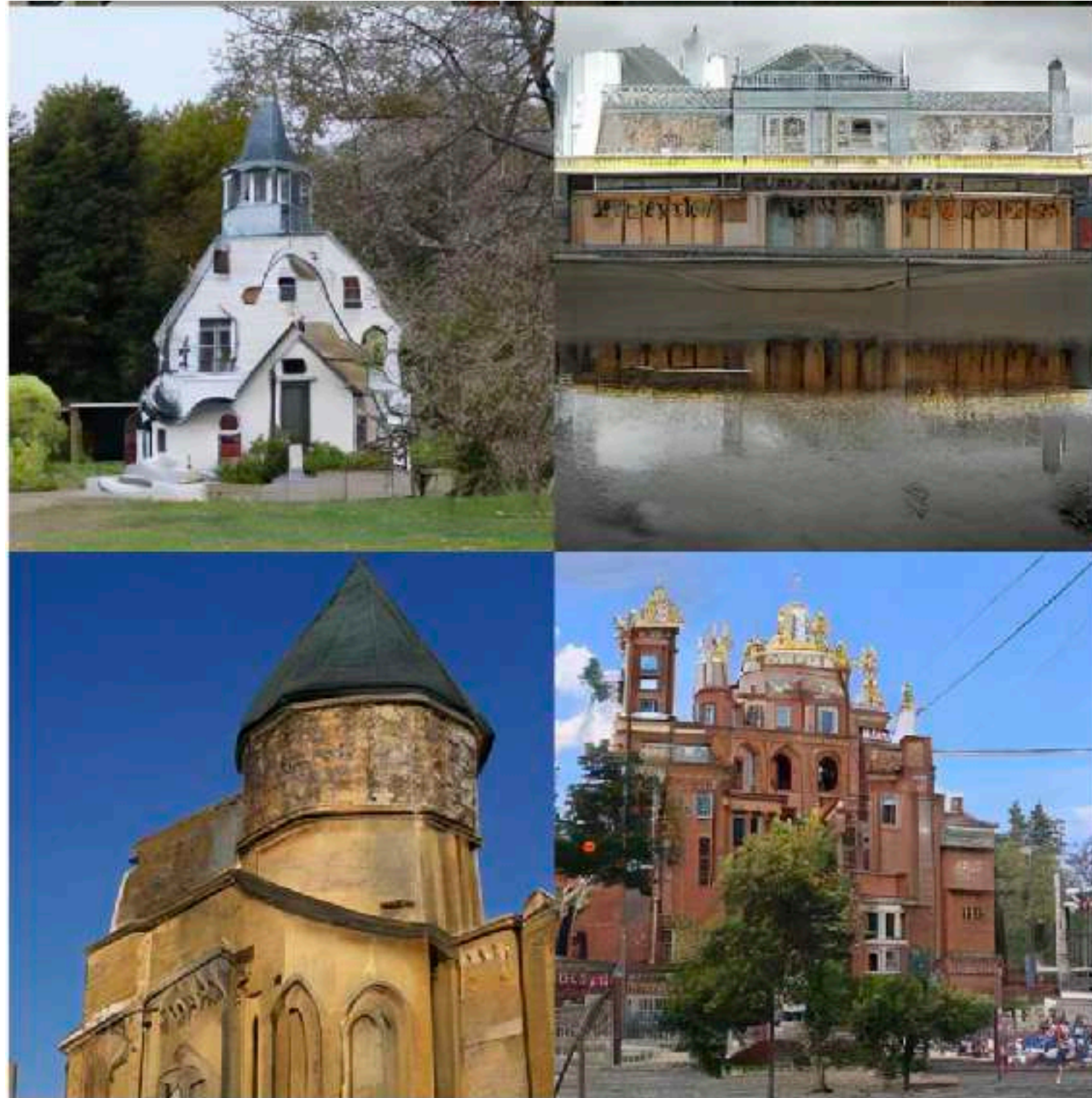


High constant CFG



Interval CFG

Interval Guidance



No CFG



High constant CFG



Interval CFG

CFG with negative conditioning

We can replace $\mathbf{v}_\theta(\mathbf{x}, \emptyset)$ with **anything** we do not want to generate

For example, we can use $\mathbf{v}_\theta(\mathbf{x}, \mathbf{c}^-)$, where $\mathbf{c}^-$ is some negative condition

$\mathbf{c}^+ = \text{"Garfield"} , \mathbf{c}^- = \text{""}$



$\mathbf{c}^+ = \text{"Garfield"} , \mathbf{c}^- = \text{"a cat"}$



CFG with negative conditioning

Negative prompt c^- example in practice

“low quality, blurry, out of focus, low resolution, pixelated, jpeg artifacts, compression artifacts, noisy, grainy, oversharpened, aliasing, banding, bad anatomy, bad proportions, deformed, disfigured, malformed limbs, mutated hands, extra fingers, missing fingers, fused fingers, extra limbs, long neck, twisted body, duplicate body parts, poorly drawn face, distorted face, asymmetrical eyes, cross-eyed, bad eyes, dead eyes, distorted pupils cropped, out of frame, cut off limbs, poorly framed, bad composition, tilted horizon watermark, 3d render, plastic skin, waxy skin, doll-like, overexposed, underexposed, harsh shadows, color banding, color bleeding, duplicate, cloned face”

AutoGuidance

Let's consider a bad version of $\mathbf{v}_\theta(\mathbf{x}, \mathbf{c})$, e.g., *early checkpoint or smaller model*

$\mathbf{v}_\theta^{bad}(\mathbf{x}, \mathbf{c})$ is trained on the *same conditioning, same distribution*
but slightly worse than the main model $\mathbf{v}_\theta^{main}(\mathbf{x}, \mathbf{c})$

$$\mathbf{v}_{AG}(\mathbf{x} | \mathbf{c}) = \mathbf{v}_\theta^{bad}(\mathbf{x}, \mathbf{c}) + w \cdot (\mathbf{v}_\theta^{main}(\mathbf{x}, \mathbf{c}) - \mathbf{v}_\theta^{bad}(\mathbf{x}, \mathbf{c})), \quad w \geq 1$$

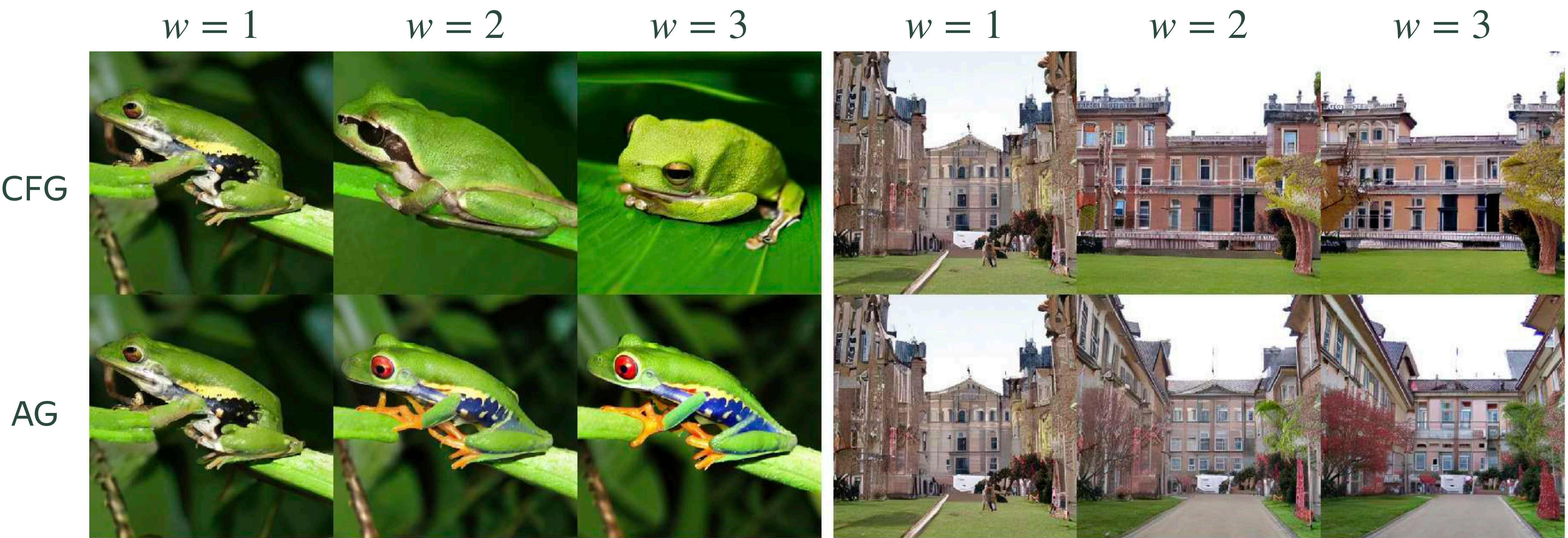
Why AG works?

$\mathbf{v}_{\theta}^{bad}(\mathbf{x}, \mathbf{c})$ is globally aligned with $\mathbf{v}_{\theta}^{main}(\mathbf{x}, \mathbf{c})$ but $\mathbf{v}_{\theta}^{bad}(\mathbf{x}, \mathbf{c})$ points to more “error” directions

Conceptual decomposition: $\mathbf{v}_{\theta}^{bad}(\mathbf{x}, \mathbf{c}) = \mathbf{v}_{\theta}^{main}(\mathbf{x}, \mathbf{c}) + \delta \cdot \mathbf{v}_{\theta}^{error}(\mathbf{x}, \mathbf{c})$

$$\mathbf{v}_{AG}(\mathbf{x} | \mathbf{c}) = \mathbf{v}_{\theta}^{bad}(\mathbf{x}, \mathbf{c}) + w \cdot (\mathbf{v}_{\theta}^{main}(\mathbf{x}, \mathbf{c}) - \mathbf{v}_{\theta}^{bad}(\mathbf{x}, \mathbf{c})) = \mathbf{v}_{\theta}^{main}(\mathbf{x}, \mathbf{c}) - \hat{w} \cdot \mathbf{v}_{\theta}^{error}(\mathbf{x}, \mathbf{c})$$

Results



Takeaways

Guidance — a **core** technique in **all** diffusion models in practice

Classifier-Free Guidance — **default** for text-to-image / video. Often, **negative prompt** instead of $\emptyset$

Interval Guidance — simple and popular idea to **increase diversity** and **reduce oversaturation**

AutoGuidance — highly promising idea but hard to find a proper guiding (“bad”) model in practice

There are **numerous** variants of guidance approaches

Different guidance methods can be combined together